

FLUSH+RELOAD: a High Resolution, Low Noise, L3 Cache Side-Channel Attack

Yuval Yarom

Katrina Falkner

The University of Adelaide

Abstract

Sharing memory pages between non-trusting processes is a common method of reducing the memory footprint of multi-tenanted systems. In this paper we demonstrate that, due to a weakness in the Intel X86 processors, page sharing exposes processes to information leaks. We present FLUSH+RELOAD, a cache side-channel attack technique that exploits this weakness to monitor access to memory lines in shared pages. Unlike previous cache side-channel attacks, FLUSH+RELOAD targets the Last-Level Cache (i.e. L3 on processors with three cache levels). Consequently, the attack program and the victim do not need to share the execution core.

We demonstrate the efficacy of the FLUSH+RELOAD attack by using it to extract the private encryption keys from a victim program running GnuPG 1.4.13. We tested the attack both between two unrelated processes in a single operating system and between processes running in separate virtual machines. On average, the attack is able to recover 96.7% of the bits of the secret key by observing a single signature or decryption round.

1 Introduction

To reduce the memory footprint of a system, the system software shares identical memory pages between processes running on the system. Such sharing can be based on the source of the page, as is the case in shared libraries [13, 26, 42]. Alternatively, the sharing can be based on actively searching and coalescing identical contents [6, 55]. To maintain the isolation between non-trusting processes, the system relies on hardware mechanisms that enforce read only or copy-on-write [13, 40] semantics for shared pages. While the processor ensures that processes cannot change the contents of shared memory pages, it sometimes fails to block other forms of inter-process interference.

One form of interference through shared pages results

from the shared use of the processor cache. When a process accesses a shared page in memory, the contents of the accessed memory location is cached. Gullasch et al. [29] describes a side channel attack technique that utilises this cache behaviour to extract information on access to shared memory pages. The technique uses the processor’s `clflush` instruction to evict the monitored memory locations from the cache, and then tests whether the data in these locations is back in the cache after allowing the victim program to execute a small number of instructions.

We observe that the `clflush` instruction evicts the memory line from all the cache levels, including from the shared Last-Level-Cache (LLC). Based on this observation we design the FLUSH+RELOAD attack—an extension of the Gullasch et al. attack. Unlike the original attack, FLUSH+RELOAD is a cross-core attack, allowing the spy and the victim to execute in parallel on different execution cores. FLUSH+RELOAD further extends the Gullasch et al. attack by adapting it to a virtualised environment, allowing cross-VM attacks.

Two properties of the FLUSH+RELOAD attack make it more powerful, and hence more dangerous, than prior micro-architectural side-channel attacks. The first is that the attack identifies access to specific memory lines, whereas most prior attacks identify access to larger classes of locations, such as specific cache sets. Consequently, FLUSH+RELOAD has a high fidelity, does not suffer from false positives and does not require additional processing for detecting access. While the Gullasch et al. attack also identifies access to specific memory lines, the attack frequently interrupts the victim process and as a result also suffers from false positives.

The second advantage of the FLUSH+RELOAD attack is that it focuses on the LLC, which is the cache level furthest from the processors cores (i.e., L2 in processors with two cache levels and L3 in processors with three). The LLC is shared by multiple cores on the same processor die. While some prior attacks do use the

F LUSH +R ELOAD :一种高分辨率、低噪声的L3缓存侧信道攻击

尤瓦尔·亚罗姆

卡特琳娜·福克纳

阿德莱德大学

摘要

在互不信任的进程之间共享内存页面是减少多租户系统内存占用的常见方法。本文证明，由于Intel X86处理器存在安全漏洞，页面共享机制会使进程面临信息泄露风险。我们提出F LUSH+R ELOAD缓存侧信道攻击技术，该技术利用这一漏洞监控共享页面内存行的访问行为。与传统缓存侧信道攻击不同，F LUSH+R ELOAD专门针对三级缓存（即具有三级缓存的处理器中的L3缓存）进行攻击。因此，攻击程序与被攻击系统无需共享执行核心即可实施攻击。

我们通过使用F LUSH+R ELOAD攻击从运行GnuPG 1.4.13的受害者程序中提取私钥，验证了该攻击的有效性。我们分别在单一操作系统中两个无关进程之间以及在独立虚拟机中运行的进程之间测试了该攻击。平均而言，通过观察单个签名或解密轮次，该攻击能够恢复密钥96.7%的位。

1 介绍

为降低系统内存占用，系统软件会在运行进程间共享相同内存页。这种共享机制既可基于页面来源（如共享库[13,26,42]），也可通过主动搜索并合并相同内容实现[6,55]。为确保互信进程间的隔离性，系统采用硬件机制强制执行共享页的只读或写时复制语义[13,40]。虽然处理器能确保进程无法修改共享内存页内容，但有时仍无法有效阻断其他形式的进程间干扰。

一种通过共享页面产生的干扰形式

这种攻击源于处理器缓存的共享使用机制。当进程访问内存中的共享页面时，被访问内存位置的内容会被缓存。古拉斯奇等人[29]提出了一种利用这种缓存行为的侧信道攻击技术，通过监控共享内存页面的访问行为来获取信息。该技术利用处理器的clflush指令将被监控的内存位置从缓存中清除，随后在允许攻击程序执行少量指令后，检测这些位置的数据是否已重新进入缓存。

我们观察到clflush指令会将内存行从所有缓存层级中驱逐，包括共享的最后级缓存（LLC）。基于这一观察，我们设计了F LUSH+R ELOAD攻击——这是Gullasch等人攻击的扩展。与原始攻击不同，F LUSH+R ELOAD是一种跨核心攻击，允许间谍和受害者在不同的执行核心上并行执行。F LUSH+R ELOAD通过将其适配到虚拟化环境中，进一步扩展了Gullasch等人攻击，允许跨虚拟机攻击。

F LUSH+R ELOAD攻击的两个特性使其比以往的微架构侧信道攻击更强大，因而也更具危险性。首先，该攻击能识别对特定内存行的访问，而大多数先前攻击仅能识别对较大类别的位置（如特定缓存组）的访问。因此，F LUSH+R ELOAD具有高保真度，不会出现误报，且无需额外处理即可检测访问。虽然Gullasch等人提出的攻击同样能识别对特定内存行的访问，但该攻击常会中断目标进程，因此也会产生误报。

F LUSH+R ELOAD攻击的第二个优势在于其针对LLC（最远离处理器核心的缓存层级，即双缓存处理器的L2和三缓存处理器的L3）进行攻击。LLC由同一处理器芯片上的多个核心共享。虽然部分先前的攻击确实利用了

LLC [47, 60], all of these attacks have a very low resolution and cannot, therefore, attain the fine granularity required, for example, for cryptanalysis.

To demonstrate the power of FLUSH+RELOAD we use it to mount an attack on the RSA [48] implementation of GnuPG [27]. We test the attack in two different scenarios. In the same-OS scenario both the spy and the victim execute as processes in the same operating system. In the cross-VM scenario, the spy and the victim execute in separate, co-located virtual machines. Both scenarios were tested in a local lab settings on otherwise idle machines.

By observing a single signing or decryption round, the attack extracts 98.7% of the bits on average in the same-OS scenario and 96.7% in the cross-VM scenario, with a worst case of 95% and 90%, respectively.

The rest of this paper is organised as follows. The next section presents background information on page sharing, cache architecture and the RSA encryption. Section 3 describes the FLUSH+RELOAD technique, followed by a description of our attack on GnuPG in Section 4. Mitigation techniques are presented in Section 5, and the related work in Section 6.

2 Preliminaries

2.1 Page Sharing

Sharing memory between processes can serve two different aims. It can be used as an inter-process communication mechanisms between two co-operating processes and it can be used for reducing memory footprint by avoiding replicated copies of identical contents. This paper focuses on the latter use.

When using *content-aware sharing*, identical pages are identified by the disk location the contents of the page is loaded from. This is the traditional form of sharing in an operating system, which is used for sharing the text segment of executable files between processes executing it and when using shared libraries [26]. Context-aware sharing has been suggested in early operating systems, such as Multics [42] and TENEX [13], and is implemented in all current major operating systems. This approach has also been suggested within the context of virtualisation hypervisors, such as Disco [15] and Satori [39].

Content-based page sharing, also called *memory de-duplication*, is a more aggressive form of page sharing. When using de-duplication, the system scans the active memory, identifying and coalescing unrelated pages with identical contents. De-duplication is implemented in the VMware ESX [54, 55] and PowerVM [17] hypervisors, and has also been implemented in Linux [6] and in Windows [33].

As memory pages can be shared between non co-operating processes, the system must protect the contents of the pages to prevent malicious processes from modifying the shared contents. To achieve this, the system maps shared pages as copy-on-write [13, 40]. Read operations on copy-on-write pages are permitted whereas write operations cause a CPU trap. The system software, which gains control of the CPU during the trap, copies the contents of the shared page, maps the copied page into the address space of the writing process and resumes the process.

While copy-on-write protects shared pages from modifications, it is not fully transparent. The delay introduced when modifying a shared page can be detected by processes, leading to a potential information leak attack. Such attacks have been implemented within virtualised environments for creating covert channels [58], for OS fingerprinting [44] and for detection of applications and data in other guests [49].

2.2 Cache Architecture

In addition to sharing memory pages, processes running on the same processor share the processor caches. Processor caches bridge the gap between the processing speed of modern processors and the data retrieval speed of the memory. Caches are small banks of fast memory in which the processor stores values of recently accessed memory cells. Due to locality of reference, recently used values tend to be used again. Retrieving these values from the cache saves time and reduces the pressure on the main memory.

Modern processors employ a cache hierarchy consisting of multiple caches. For example, the cache hierarchy of the Core i5-3470 processor, shown in Fig. 1, consists of three cache levels: L1, L2 and L3.

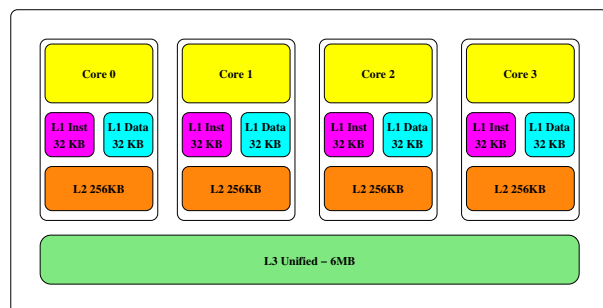


Figure 1: Intel Ivy Bridge Cache Architecture

The Core i5-3470 processor has four processing units called *cores*. Each core has a 64KB L1 cache, divided into a 32KB data cache and a 32KB instruction cache. Each core also has a 256KB L2 cache. The four cores

LLC [47,60], 所有这些攻击手段的分辨率极低, 因此无法达到密码分析等场景所需的精细粒度。

为验证F LUSH+R ELOAD的攻击能力, 我们利用该技术对GnuPG[27]的RSA[48]实现发起攻击。在两种不同场景下进行测试: 同一操作系统场景中, 间谍程序与目标程序作为同一操作系统中的进程运行; 跨虚拟机场景中, 两者分别运行于独立但共址的虚拟机中。所有测试均在本地实验室环境中进行, 所用机器处于空闲状态。

通过观察单次签名或解密轮次, 该攻击在相同操作系统场景下平均可提取98.7%的比特, 在跨虚拟机场景下平均可提取96.7%的比特, 最坏情况下分别为95%和90%。

本文的其余部分组织如下。下一节介绍了页面共享、缓存架构和RSA加密的背景信息。第3节描述了F LUSH+R ELOAD技术, 然后在第4节描述了我们针对GnuPG的攻击。缓解技术在第5节中提出, 相关工作在第6节中。

2 前言

2.1 页面共享

进程间共享内存可以实现两个不同的目的:一是作为两个合作进程之间的进程间通信机制, 二是通过避免相同内容的重复拷贝来减少内存占用。

在使用内容感知共享时, 系统会通过页面内容的加载磁盘位置来识别相同页面。这是操作系统中传统的共享形式, 用于在执行可执行文件的进程之间共享文本段, 以及在使用共享库时实现[26]。上下文感知共享的概念最早出现在Multics[42]和 TENEX [13]等早期操作系统中, 并已应用于所有主流操作系统。该技术方案同样被虚拟化管理程序提出, 例如Disco[15]和Satori[39]等虚拟化管理程序。

基于内容的页面共享, 也称为内存去重, 是一种更激进的页面共享形式。使用去重时, 系统会扫描活动内存, 识别并合并内容相同的无关页面。VMware ESX [54,55]和PowerVM[17]虚拟机监控程序中实现了去重功能, Linux[6]和Windows[33]中也实现了该功能。

由于内存页面可在非协作进程间共享, 系统必须保护页面内容以防止恶意进程篡改共享数据。为此, 系统采用写时复制机制[13,40]来映射共享页面。写时复制页面允许读取操作, 而写入操作会触发CPU陷阱。当系统软件在陷阱期间接管CPU控制权时, 会先复制共享页面内容, 将复制后的页面映射到写入进程的地址空间, 随后恢复该进程的运行。

虽然写时复制技术能保护共享页面免受修改, 但其防护并非完全透明。当修改共享页面时产生的延迟可能被进程检测到, 从而导致潜在的信息泄露攻击。此类攻击已在虚拟化环境中被用于创建隐蔽通道[58]、进行操作系统指纹识别[44], 以及检测其他客户机中的应用程序和数据[49]。

2.2 缓存体系结构

除了共享内存页面, 同一处理器上运行的进程还会共享处理器缓存。处理器缓存的作用是弥合现代处理器处理速度与内存数据读取速度之间的差距。缓存是处理器用来存储最近访问过的内存单元数据的小型高速存储器。由于引用的局部性, 最近使用的数据往往会被再次调用。从缓存中快速读取这些数据不仅能节省时间, 还能减轻主内存的负担。

现代处理器采用由多个缓存组成的缓存层次结构。例如, 如图1所示的Core i5-3470处理器的缓存层次结构由三个缓存层级组成: L1、L2和L3。

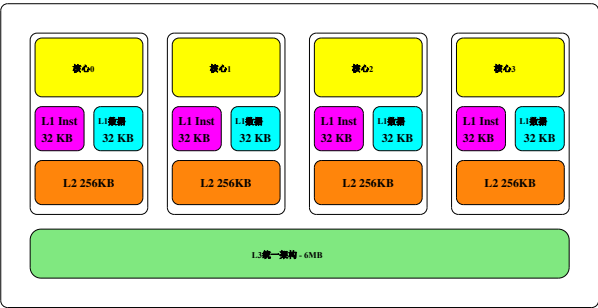


图1: 英特尔Ivy Bridge缓存架构

酷睿i5-3470处理器有四个处理单元, 称为核心。每个核心都有64KB的L1缓存, 分为32KB的数据缓存和32KB的指令缓存。每个核心还配备256KB的L2缓存。四个核心

share a 6MB L3 cache, also known as the Last-Level Cache, or LLC.

The unit of memory in a cache is a *line* which contains a fixed number of bytes. A cache consists of multiple *cache sets* each of which stores a fixed number of cache lines. The number of cache lines in a set is the *cache associativity*. Each memory line can be cached in any of the cache lines of a single cache set. The size of cache lines in the Core i5-3470 processor is 64 bytes. The L1 and L2 caches are 8-way associative and the L3 cache is 12-way associative.

An important feature of the LLC in modern Intel processors is that it is an inclusive cache. That is, the LLC contains copies of all of the data stored in the lower cache levels. Consequently, flushing or evicting data from the LLC also remove said data from all other cache levels of the processor. Our attack exploits this cache behaviour.

Retrieving data from memory or from cache levels closer to memory takes longer than retrieving it from cache levels closer to the core. This difference in timing has been exploited for side-channel attacks. Side-channel attacks target information that an implementation of an algorithm leaks through its interaction with its environment. To exploit the timing difference, an attacker sets the cache to a known state prior to a victim operation. It can, then, use one of two methods to deduce information on the victim's operation [43]. The first method is measuring the time it takes for the victim to execute the operation. As this time depends on the state of the cache when the victim starts the operation, the attacker can deduce the cache sets accessed by the victim and, therefore, learn information on the victim [5, 9, 57]. The second approach is for the attacker to measure the time it takes for the attacker to access data after the victim's operation. This time is dependent on the cache state prior to the victim operation as well as on the changes the victim operation caused in the cache state [1, 2, 4, 14, 19, 47, 61].

Most prior work on cache side-channel attacks relies on the victim and spy executing within the same processing core. One reason for that is that many of the attacks suggested require the victim to be stopped while the spy performs the attack. To that aim, the attack is combined with an attack on the scheduler that allows the spy process to interrupt and block the victim.

Another reason for attacking within the same core is that the attacks focus on the L1 cache level, which is not shared between cores. The large size of the LLC hinders attacks both because setting it to a known state takes longer than with smaller caches and because the virtual memory used by the operating system masks the mapping of memory addresses to cache sets. Furthermore, as most of the memory activity occurs at the L1 cache level, less information can be extracted from LLC activ-

ity. Some prior works do use the LLC as an information leak channel [46, 47, 60]. However, due to the cache size, these channels have a low bandwidth.

We now proceed to describe the RSA encryption.

2.3 RSA

RSA [48] is a public-key cryptographic system that supports encryption and signing. Generating an encryption system requires the following steps:

- Randomly selecting two prime numbers p and q and calculating $n = pq$.
- Choosing a public exponent e . GnuPG uses $e = 65537$.
- Calculating a private exponent $d \equiv e^{-1} \pmod{(p-1)(q-1)}$.

The generated encryption system consists of:

- The public key is the pair (n, e) .
- The private key is the triple (p, q, d) .
- The encrypting function is $E(m) = m^e \bmod n$.
- The decrypting function is $D(c) = c^d \bmod n$.

CRT-RSA is a common optimisation for the implementation of the decryption function. It splits the secret key d into two parts $d_p = d \bmod (p-1)$ and $d_q = d \bmod (q-1)$, computes two parts of the message: $m_p = c^{d_p} \bmod p$ and $m_q = c^{d_q} \bmod q$. m is then computed from m_p and m_q using Garner's formula [25]:

$$h = (m_p - m_q)(q^{-1} \bmod p) \bmod p$$

$$m = m_q + hq$$

To compute the encryption and decryption functions, GnuPG versions before 4.1.14 and the related libgcrypt before version 1.5.3 use the square-and-multiply exponentiation algorithm [28]. Square-and-multiply computes $x = b^e \bmod m$ by scanning the bits of the binary representation of the exponent e . Given a binary representation of e as $2^{n-1}e_{n-1} + \dots + 2^0e_0$, square-and-multiply calculates a sequence of intermediate values $x_{n-1}, \dots, x_0$ such that $x_i = b^{\lfloor e/2^i \rfloor} \bmod m$ using the formula $x_{i-1} = x_i^2 b^{e_{i-1}}$. Figure 2 shows a pseudo-code implementation of square-and-multiply.

As can be seen from the implementation, computing the exponent consists of sequence of Square and Multiply operations, each followed by a Modulo Reduce. This sequence corresponds directly with the bits of the exponent. Each occurrence of Square-Reduce-Multiply-Reduce within the sequence corresponds to a bit whose value is 1. Occurrences of Square-Reduce that are not

共享6MB的三级缓存（L3 Cache），该缓存也称为最后一级缓存（LLC）。

缓存中的存储单元称为行，每个行包含固定数量的字节。缓存由多个缓存组构成，每个缓存组存储固定数量的缓存行。缓存组中的缓存行数量即为缓存关联性。每个内存行可被缓存在单个缓存组的任意缓存行中。Core i5-3470处理器的缓存行大小为64字节，其中L1和L2缓存采用8路关联，L3缓存则为12路关联。

现代英特尔处理器的LLC缓存具有一个关键特性——其具备全缓存特性。这意味着LLC缓存中存储着下层缓存层级的所有数据副本。因此，当从LLC缓存中刷新或驱逐数据时，这些数据也会被清除出处理器的所有其他缓存层级。我们的攻击正是利用了这一缓存特性。

从内存或更接近内存的缓存层级获取数据，比从更靠近核心的缓存层级获取数据耗时更长。这种时间差异已被用于侧信道攻击。侧信道攻击针对算法实现通过与环境交互时泄露的信息。攻击者会利用这种时间差，通过在受害者执行操作前将缓存设置为已知状态，进而采用两种方法之一来推断受害者操作的信息[43]。第一种方法是测量受害者执行操作所需的时间。由于该时间取决于受害者开始操作时的缓存状态，攻击者可以推断出受害者访问的缓存设置，从而获取受害者信息[5,9,57]。第二种方法是攻击者测量自身在受害者操作后访问数据所需的时间。该时间既取决于受害者操作前的缓存状态，也取决于受害者操作对缓存状态造成的改变[1,2,4,14,19,47,61]。

现有缓存侧信道攻击研究大多基于攻击者与受害者共用同一处理器核心的场景。这主要是因为多数攻击方案需要在停止被攻击进程的同时实施。为此，攻击者通常会联合调度器攻击，使被攻击进程陷入中断并被阻塞。

攻击同一核心的另一个原因在于，攻击主要针对L1缓存层级——该层级在不同核心间不共享。LLC（乱序缓存）的庞大容量会阻碍攻击实施，一方面是因为将其设置为已知状态所需时间比小容量缓存更长，另一方面由于操作系统使用的虚拟内存会掩盖内存地址到缓存组的映射关系。此外，由于大部分内存活动发生在L1缓存层级，从LLC活动中可提取的信息量相对较少。

某些先前的研究确实将LLC（链路层控制）用作信息泄露通道[46,47,60]。然而，由于缓存容量的限制，这些通道的带宽较低。

接下来我们将详细说明RSA加密算法。

2.3 RSA

RSA[48]是一种支持加密和签名的公钥密码系统。

生成加密系统需遵循以下步骤：

- 随机选择两个素数 p 和 q ，并计算 $n = pq$ 。
- 选择公钥 e 。GnuPG 使用 $e = 65537$ 。
- 计算私有指数 $d \equiv e^{-1} \pmod{(p-1)(q-1)}$ 。

生成的加密系统由以下部分组成：

- 公钥是对 $(n; e)$ 。
- 私钥是三元组 $(p; q; d)$ 。
- 加密函数为 $E(m) = m^e \bmod n$ 。
- 解密函数为 $D(c) = c^d \bmod n$ 。

CRT-RSA是实现解密函数的常见优化方案。它将密钥 d 拆分为两部分 $d_p = d \bmod (p-1)$ 和 $d_q = d \bmod (q-1)$ ，并计算消息的两部分： $m_p = c^{d_p} \bmod p$ 和 $m_q = c^{d_q} \bmod q$ 。 m 随后通过加纳公式[25]从 m_p 和 m_q 计算得出：

$$\begin{aligned} h &= (m_p - m_q)(q^{-1} \bmod p) \bmod p \\ m &= m_q + hq \end{aligned}$$

为计算加密和解密函数，GnuPG 4.1.14 之前版本及相关 libgcrypt 1.5.3 之前版本采用平方乘法指数运算算法[28]。平方乘法通过扫描指数 e 的二进制表示的位来计算 $x = b^e \bmod m$ 。给定 e 的二进制表示为 $2^{n-1}e_{n-1} + \dots + 2^0e_0$ ，平方乘法计算一系列中间值 $x_{i-1}; \dots; x_0$ ，使得 $x_i = b^{[e/2^i]} \bmod m$ ，使用公式 $x_{i-1} = x_i^2 \bmod m$ 。图2展示了平方乘法的伪代码实现。

从实现过程可见，指数的计算由一系列平方运算和乘法运算组成，每个运算后均需进行模运算。该序列直接对应指数的比特位。序列中每次出现的平方-模运算-乘法-模运算组合，对应一个值为1的比特位。而未出现的平方-模运算则

```

1 function exponent( $b, e, m$ )
2 begin
3    $x \leftarrow 1$ 
4   for  $i \leftarrow |e| - 1$  downto 0 do
5      $x \leftarrow x^2$ 
6      $x \leftarrow x \bmod m$ 
7     if ( $e_i = 1$ ) then
8        $x \leftarrow xb$ 
9        $x \leftarrow x \bmod m$ 
10    endif
11  done
12  return  $x$ 
13 end

```

Figure 2: Exponentiation by Square-and-Multiply

followed by a Multiply correspond to bits whose values are 0. Consequently, a spy process that can trace the execution of the square-and-multiply exponentiation algorithm can recover the exponent.

As GnuPG uses the CRT-RSA optimisation, the spy process can only hope to extract d_p and d_q . However, for an arbitrary message m , $(m - m^{ed_p})$ is a multiple of p . Hence, knowing d_p (and, symmetrically, d_q) is sufficient for factoring n and breaking the encryption [16].

3 The FLUSH+RELOAD Technique

The FLUSH+RELOAD technique is a variant of PRIME+PROBE [51] that relies on sharing pages between the spy and the victim processes. With shared pages, the spy can ensure that a specific memory line is evicted from the whole cache hierarchy. The spy uses this to monitor access to the memory line. The attack is a variation of the technique suggested by Gullasch et al. [29], which include adaptations for use in multi-core and in virtualised environments.

A round of attack consists of three phases. During the first phase, the monitored memory line is flushed from the cache hierarchy. The spy, then, waits to allow the victim time to access the memory line before the third phase. In the third phase, the spy reloads the memory line, measuring the time to load it. If during the wait phase the victim accesses the memory line, the line will be available in the cache and the reload operation will take a short time. If, on the other hand, the victim has not accessed the memory line, the line will need to be brought from memory and the reload will take significantly longer. Figure 3 (A) and (B) show the timing of the attack phases without and with victim access.

As shown in Fig. 3 (C), the victim access can overlap the reload phase of the spy. In such a case, the victim access will not trigger a cache fill. Instead, the victim will use the cached data from the reload phase. Consequently, the spy will miss the access.

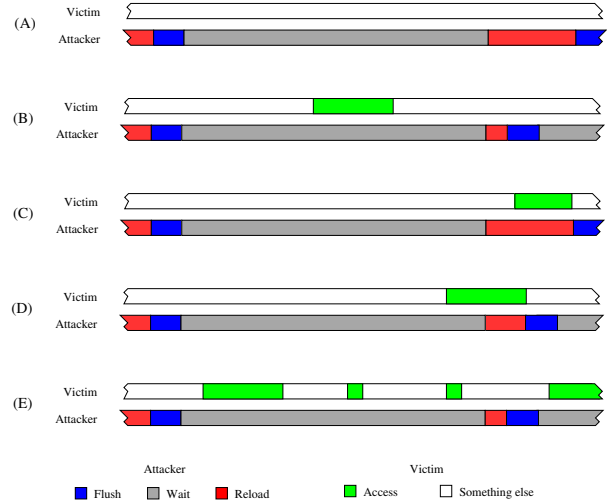


Figure 3: Timing of FLUSH+RELOAD. (A) No Victim Access (B) With Victim Access (C) Victim Access Overlap (D) Partial Overlap (E) Multiple Victim Accesses

A similar scenario is when the reload operation partially overlaps the victim access. In this case, depicted in Fig. 3 (D), the reload phase starts while the victim is waiting for the data. The reload benefits from the victim access and terminates faster than if the data has to be loaded from memory. However, the timing may still be longer than a load from the cache.

As the victim access is independent of the execution of the spy process code, increasing the wait period reduces the probability of missing the access due to an overlap. On the other hand, increasing the wait period reduces the granularity of the attack.

One way to improve the resolution of the attack without increasing the error rate is to target memory accesses that occur frequently, such as a loop body. The attack will not be able to discern between separate accesses, but, as Fig. 3 (E) shows, the likelihood of missing the loop is small.

Several processor optimisations may result in false positives due to speculative memory accesses issued by the victim's processor [34]. These optimisations include data prefetching to exploit spatial locality and speculative execution [52]. When analysing the attack results, the attacker must be aware of these optimisations and develop strategies to filter them.

Our implementation of the attack is in Figure 4. The code measures the time to read the data at a memory address and then evicts the memory line from the cache. This measurement is implemented by the inline assembly code within the `asm` command.

The assembly code takes one input, the address, which is stored in register `%ecx`. (Line 16.) It returns the time to read this address in the register `%eax` which is stored


```

1 函数指数 ( $b, e, m$ )
2 开始
3  $x \leftarrow 1$ 
4 用于  $i \leftarrow jej - 1$  从 1 递减至 0
5  $x \leftarrow x^2$ 
6  $x \leftarrow x \bmod m$ 
7 如果 ( $ei = 1$ ) 则
8  $x \leftarrow xb$ 
9  $x \leftarrow x \bmod m$ 
10 endif
11 完成
12 返回  $x$ 
13 端

```

图2：平方乘法的指数运算

随后的乘法运算对应于值为0的位。因此，能够追踪平方-乘法指数运算算法执行过程的间谍进程可恢复指数。

由于GnuPG使用CRT-RSA优化，间谍进程只能尝试提取 d_p 和 d_q 。然而，对于任意消息 m ，($m - med_p$)是 p 的倍数。因此，只要知道 d_p （对称地， d_q ），就足以分解 n 并破解加密[16]。

3 F LUSH +R ELOAD 技术

F LUSH +R ELOAD 技术是P RIME +P ROBE [51]的一种变体，其核心在于间谍进程与目标进程共享内存页。通过共享内存页，间谍进程能够确保特定内存行从整个缓存层级中被清除。该技术利用这一特性监控内存行的访问行为。此攻击方法是对Gullasch等人[29]提出技术的改进版本，其中包含针对多核环境和虚拟化环境的适配方案。

一轮攻击包含三个阶段。第一阶段，监控的内存行会被从缓存层级中清除。此时间课程程序会等待，为受害者争取在第三阶段前访问该内存行的时间。第三阶段，间谍程序重新加载内存行并测量加载时间。若在等待阶段受害者访问过该内存行，该行将存在于缓存中，重新加载操作耗时较短；反之，若受害者未访问过该内存行，则需从内存中获取，重新加载操作耗时显著增加。图3(A)和(B)分别展示了无受害者访问与有受害者访问时攻击各阶段的时间分布。

如图3(C)所示，受害者访问可能与间谍的重新加载阶段重叠。此时，受害者访问不会触发缓存填充，而是使用重新加载阶段的缓存数据。因此，间谍将无法捕获该访问。

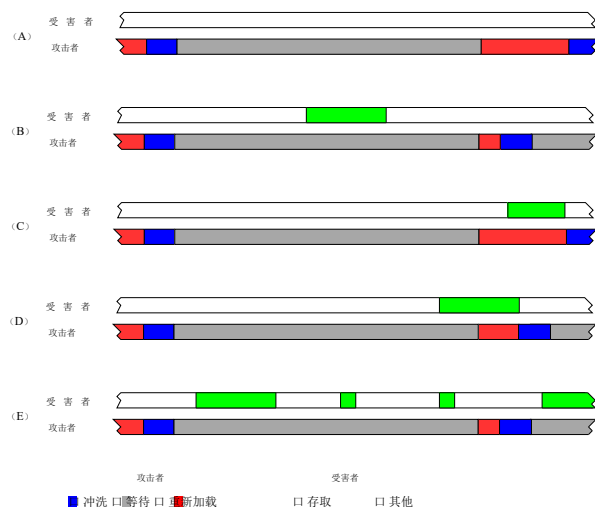


图3：F LUSH+R ELOAD的时间分布。(A)无受害者访问(B)有受害者访问(C)受害者访问重叠(D)部分重叠(E)多次受害者访问

类似的情形还出现在重载操作与目标访问部分重叠时。如图3(D)所示，此时重载阶段在目标等待数据期间启动。由于受益于目标访问，重载操作比从内存加载数据更快完成。不过，其耗时仍可能超过从缓存加载数据。

由于攻击者访问与间谍进程的执行无关，增加等待时间可以减少攻击者因攻击与间谍进程的重叠而错过访问的机会，但也会降低攻击的粒度。

若要在不增加错误率的前提下提升攻击的分辨率，可针对频繁发生的内存访问（如循环体）进行攻击。虽然攻击者无法区分不同访问行为，但如图3(E)所示，漏检循环体的概率极低。

多种处理器优化技术可能因攻击目标处理器的推测性内存访问[34]而产生误报。这些优化技术包括利用空间局部性的数据预取和推测执行[52]。在分析攻击结果时，攻击者必须意识到这些优化技术，并制定相应的过滤策略。

图4展示了我们实现的攻击方案。该代码通过测量读取内存地址数据所需的时间，将该内存行从缓存中移除。此测量操作由asm命令内的内联汇编代码实现。

汇编代码接收一个输入地址，该地址存储于寄存器%ecx中（第16行）。它返回寄存器%eax中存储的该地址的读取时间。

from the cache. Loads longer than the threshold are presumed to be served from the memory, indicating no access to the memory line.

The threshold used in the attack is system dependent. To find the threshold for our test systems, we used the measurement code of the probe in Listing 4 to measure load times from memory and from the L1 cache level. (To measure the L1 times we removed the `clflush` instruction in line 14.) The results of 100,000 measurements of each on an HP Elite 8300 with an i5-3470 processor, running CentOS 6.5 are presented in Figure 5.

Virtually all loads from the L1 cache measure 44 cycles. (Note that this measure includes an overhead for the `rdtsc` and the fence instructions and is, therefore, much longer than a single load instruction.) Loads from memory show less constant timing. Over 98% of those take between 270 and 290 cycles. The rest are mostly spread around 880 cycles with about 200 loads measured 1140–1175 cycles. No loads from memory measured less than 200 cycles.

The timings of load operations depend on both the system architecture and the software environment. For example, on a Dell PowerEdge T420 with Xeon E5-2430 processors, loads from L1 take between 33 and 43 cycles and loads from memory take around 230 cycles. On the same architecture, within a KVM [37] guest, about 0.02% of the loads from memory take over 6,000 cycles. We believe these are caused by hypervisor activity.

The L1 measurements underestimate the probe time for data that the victim accesses. In an attack, data the victim accesses is read from the L3 cache. Intel documentation [34] states that the difference is between 22 and 39 cycles. Based on the measurement results and the Intel documentation we set the threshold to 120 cycles.

To use the FLUSH+RELOAD technique the spy and the victim processes need to share both the cache hierarchy and memory pages. In a non-virtualised environment, to share the cache hierarchy, the attacker needs the ability to execute software on the victim machine. The attacker, however, does not need elevated privileges on the victim machine. For a virtualised environment, the attacker needs access to a guest co-located on the same host as the victim guest. Techniques for achieving co-location are described by Ristenpart et al. [47]. Identifying the OS and software version in co-resident guests has been dealt with in past research [44, 49].

For sharing memory pages in system that use content-aware sharing, the attacker needs read access to the attacked executable or shared libraries. In systems that support de-duplication the attacker needs access to a copy of the attacked files. De-duplication will coalesce pages from these copies with pages from the attacked files.

4 Attacking GnuPG

In this section we describe how we use the FLUSH+RELOAD technique to extract the components of the private key from the GnuPG implementation of RSA.

We tested the attack on two hardware platforms: an HP Elite 8300, which features an Intel Core i5-3470 processor and 8GB DDR3-1600 memory and a Dell PowerEdge T420, with two Xeon E5-2430 processors and 32GB DDR3-1333 memory. On each hardware platform we experimented with two scenarios. The same-OS scenario tests the attack between two unrelated processes in the same operating system while the cross-VM scenario demonstrates that the attack works across the virtual machine isolation boundary in virtualised environments.

The same-OS tests use CentOS 6.5 Linux running on the hardware. The spy and the victim execute as two processes within that system. To achieve sharing, the spy `mmaps` the victim’s executable file into the spy’s virtual address space. As the Linux loader maps executable files into the process when executing them, the spy and the victim share the memory image of the mapped file. On the Dell machine we set the CPU affinity of the processes to ensure that both the victim and the spy execute on the same physical processor. We do let the processes float between the cores of the processor.

For the cross-VM scenario we used two different hypervisors: VMware ESXi 5.1 on the HP machine and Centos 6.5 with KVM on the Dell machine. In each hypervisor we created two virtual machines, one for the victim and the other for the spy. The virtual machines run CentOS 6.5 Linux. In this scenario, the spy `mmaps` a copy of the victim’s executable file. Sharing is achieved through the page de-duplication mechanisms of the hypervisors. As in the same-OS scenario, on the Dell machine we set the CPU affinity of the virtual machines to ensure execution on the same physical processor.

When a pages is shared, all of the page entries in the virtual address spaces of the sharing processes map to the same physical page. As the LLC is physically tagged, entries in the cache depend only on the physical address of the shared page with no dependency on the virtual addresses in which the page is mapped. Consequently, we do not need to take care of the virtual to physical address mapping and the attack is oblivious to some diversification techniques, such as Address Space Layout Randomization (ASLR) [50].

The approach we take is to trace the execution of the victim program. For that, the spy program applies the FLUSH+RELOAD technique to memory locations within the victim’s code segment. This, effectively, places probes within the victim program that are triggered whenever the victim executes the code in the probed memory lines. Tracing the execution allows the

从缓存加载。加载时间超过阈值的文件默认从内存加载，表明未访问该内存行。

攻击所用阈值具有系统依赖性。为确定测试系统的阈值，我们采用清单4中探针的测量代码，分别测量内存和一级缓存（L1 cache）层级的加载时间（测量L1时间时移除了第14行的clflush指令）。图5展示了在搭载i5-3470处理器、运行CentOS 6.5的HP Elite 8300服务器上，对每种情况进行10万次测量的结果。

L1缓存的加载操作几乎都耗时44个周期。（需注意，该测量值包含rdtsc和fence指令的额外开销，因此远超单次加载指令的耗时。）内存加载的时序稳定性较差，超过98%的加载操作耗时在270至290周期之间。其余操作主要分布在880周期范围内，约200次加载耗时介于1140至1175周期。未检测到任何低于...的内存加载操作。200个周期。

负载操作的执行时序取决于系统架构和软件环境。例如，在搭载至强E5-2430处理器的戴尔PowerEdge T420服务器上，L1缓存加载耗时33至43个周期，内存加载耗时约230个周期。在同一架构的KVM [37]客户机中，约0.02%的内存加载耗时超过6000个周期。我们认为这些现象源于虚拟机管理程序的活动。

L1缓存测量会低估受害者访问数据的探测时间。在攻击过程中，受害者访问的数据是从L3缓存读取的。英特尔文档[34]指出，该差异介于22至39个周期之间。根据测量结果和英特尔文档，我们将阈值设定为120个周期。

要使用F LUSH +R ELOAD 技术，间谍程序和目标程序需要共享缓存层次结构和内存页面。在非虚拟化环境中，攻击者需要在目标机器上执行软件才能共享缓存层次结构，但无需获得该机器的高权限。对于虚拟化环境，攻击者需要访问与目标虚拟机同宿主的客户机。Ristenpart等人[47]详细阐述了实现同宿主的技术方案。关于如何识别同宿主客户机中的操作系统和软件版本，已有研究[44,49]进行了探讨。

在采用内容感知共享机制的系统中，攻击者若要共享内存页面，需获取被攻击可执行文件或共享库的读取权限。对于支持去重功能的系统，攻击者则需获取被攻击文件的副本。通过去重处理，系统会将这些副本中的页面与被攻击文件的页面进行合并。

4 攻击GnuPG

本节将介绍如何运用F LUSH +R ELOAD技术，从GnuPG实现的RSA算法中提取私钥的各个组成部分。

我们针对两种硬件平台进行了攻击测试：一台配备英特尔酷睿i5-3470处理器和8GB DDR3-1600内存的惠普Elite 8300，以及一台搭载双至强E5-2430处理器和32GB DDR3-1333内存的戴尔PowerEdge T420。在每种硬件平台上，我们分别测试了两种场景：同一操作系统场景用于验证攻击在相同操作系统内两个无关进程间的传播效果，而跨虚拟机场景则证明该攻击能在虚拟化环境的虚拟机隔离边界之外有效实施。

同一操作系统测试使用运行在硬件上的CentOS 6.5 Linux系统。间谍程序和目标程序作为该系统内的两个进程运行。为实现共享，间谍程序将目标程序的可执行文件映射到自身的虚拟地址空间。由于Linux加载器在执行可执行文件时会将其映射到进程，因此间谍程序和目标程序共享该映射文件的内存映像。在戴尔机器上，我们设置了进程的CPU亲和性，确保目标程序和间谍程序在同一物理处理器上运行。同时允许这两个进程在处理器核心之间浮动运行。

在跨虚拟机场景中，我们采用了两种不同的虚拟机管理程序：惠普服务器运行VMware ESXi 5.1，戴尔服务器运行CentOS 6.5 KVM。在每种虚拟机管理程序中，我们分别创建了两个虚拟机——一个用于目标系统，另一个用于间谍系统。这两个虚拟机均运行CentOS 6.5 Linux系统。在此场景中，间谍系统会映射目标系统的可执行文件副本。通过虚拟机管理程序的页面去重机制实现资源共享。与同一操作系统场景类似，我们在戴尔服务器上设置了虚拟机的CPU亲和性，确保它们在同一个物理处理器上运行。

当页面被共享时，共享进程虚拟地址空间中的所有页面条目都会映射到同一物理页面。由于LLC采用物理标签机制，缓存条目仅依赖共享页面的物理地址，而无需考虑该页面的虚拟地址映射。因此，我们无需处理虚拟地址到物理地址的映射关系，且该攻击对某些多样化技术（如地址空间布局随机化（ASLR）[50]）具有免疫性。

我们采用的方法是追踪被监控程序的执行过程。为此，监控程序会将F LUSH+R ELOAD技术应用于被监控程序代码段内的内存地址。这实际上是在程序中植入了探针，当程序执行被监控内存行中的代码时，这些探针就会被触发。通过追踪执行过程，我们能够

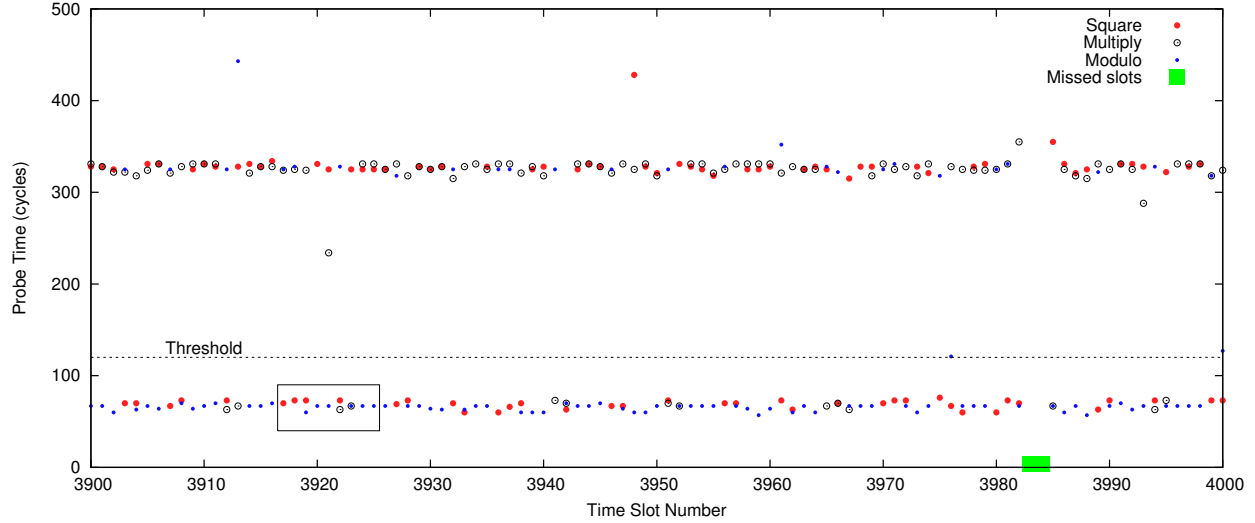


Figure 6: Time measurements of probes

spy program to infer the internal state of the victim program.

To implement the trace, the spy program divides time into fixed slots of 2,500 cycles each. In each slot it probes one memory line of the code of each of the square, multiply and modulo reduce calculations. To increase the chance of a probe capturing the access, we selected memory lines that are executed frequently during the calculation. Furthermore, to reduce the effect of speculative execution, we avoided memory lines near the beginning of the respective functions. After probing the memory lines, the spy program flushes the lines from the cache and busy waits to the end of the time slot.

We used the default build of the gpg program, which includes optimisation at -O2 level and which leaves the debugging symbols in the executable. We use the debugging symbols to facilitate the mapping of source code lines to memory addresses. In most distributions, the GnuPG executable is stripped and does not include these symbols. Attacks against stripped executables would require some reverse engineering [20] to recover this mapping. As the debugging symbols are not loaded in run time, these do not affect the victim's performance.

Measurement times for 100 time slots of the GnuPG signing with a 2,048 bit key are displayed in Figure 6. In each time slot, the spy flushes and then measures the time to read the memory lines in the Square, Multiply and Reduce functions. Measurements under the threshold indicate victim access to the respective memory lines. The exponentiations for signing takes a total of 15,690 slots or about 18ms. The CRT components used for exponentiation are 1,022 and 1,023 bits long.

Figure 7 is an enlarged view of the boxed section

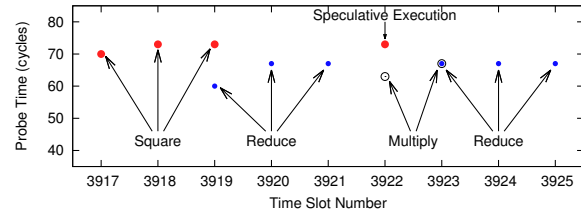


Figure 7: Section of Fig. 6

in Fig. 6. As the displayed area is below the threshold, the diagram only displays the memory lines that were retrieved from the cache, showing the activity of the GnuPG encryption. The steps of the exponentiation are clearly visible in the diagram. For example, between time slots 3,917 and 3,919 the victim was calculating a square, Time slots 3,919–3,921 are for modulo reduce calculation, multiplication in slots 3,922–3,923, and another modulo reduce in 3,923–3,925. A sequence of Square-Reduce-Multiply-Reduce indicates that during these time slots the victim was processing a set bit.

Figure 7 also demonstrates the effects of speculative execution. To improve performance, the processors tries to predict future behaviour of the program. When predicting the behaviour of the test of the bit value (Line 7 in Fig. 2), the processor does not know the value of the bit. Instead of waiting for the value to be calculated, the processor speculates that the bit might be clear and starts bringing memory lines required for the square calculation into the cache. As a result, cache lines that are part of the square calculation in Line 5 are brought into the cache, and are captured by the spy.

We have witnessed speculative execution on both the

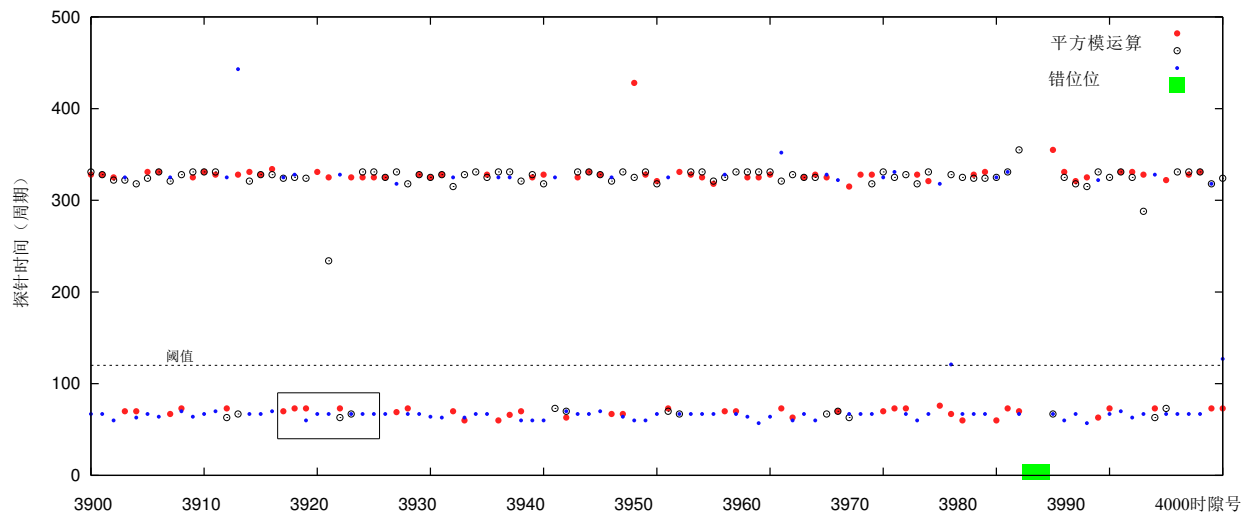


图6：探针时间测量

用于推断目标程序内部状态的间谍程序。

为实现跟踪功能，监控程序将时间划分为固定时长的2500周期时段。每个时段内，程序会逐行扫描平方运算、乘法运算和模约简运算的代码内存行。为提高捕获访问行为的概率，我们特意选取了计算过程中高频执行的内存行。同时，为降低推测执行的影响，程序刻意避开函数开头附近的内存行。完成内存行扫描后，监控程序会将这些行从缓存中清除，并通过忙等待机制持续监控至该时段结束。

我们采用gpg程序的默认构建版本，该版本包含-O 2级别的优化，并保留可执行文件中的调试符号。这些调试符号用于辅助将源代码行映射到内存地址。在大多数发行版中，GnuPG可执行文件经过反编译处理，不包含这些符号。针对反编译后的可执行文件发起攻击时，需要借助逆向工程[20]才能恢复这种映射关系。由于调试符号在运行时不会被加载，因此不会影响受害者的性能表现。

图6展示了使用2048位密钥进行GnuPG签名时100个时隙的测量时间。每个时隙中，间谍程序会先清空内存，然后测量平方、乘法和归约函数中内存行的读取时间。低于阈值的测量结果表明受害者已访问相应内存行。签名运算的指数运算总共需要15690个时隙，约18毫秒。用于指数运算的CRT组件长度分别为1022位和1023位。

图7为框选区域的放大视图

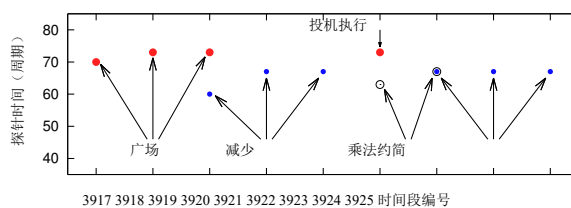


图7：图6的截面图

如图6所示，由于显示区域低于阈值，该图表仅展示从缓存中检索的存储行，呈现GnuPG加密算法的运行状态。图表中清晰可见指数运算的步骤：例如在3,917至3,919时间槽期间，受害者正在计算平方运算；3,919-3,921时间槽用于模约简运算；3,922-3,923时间槽进行乘法运算；3,923-3,925时间槽再次进行模约简运算。“平方-约简-乘法-约简”的运算序列表明，在这些时间槽期间，受害者正在处理一组比特。

图7还展示了推测执行的影响。为了提升性能，处理器会尝试预测程序的未来行为。当预测位值测试（图2中第7行）的行为时，处理器并不知道该位的具体值。处理器没有等待计算结果，而是推测该位可能是空位，并开始将平方运算所需的内存行加载到缓存中。因此，第5行中参与平方运算的缓存行被加载到缓存中，这些数据被监控器捕获。

我们已目睹双方都出现了投机性执行行为

HP and the Dell machines. Moving the probes to cache lines closer to the end of the probed functions eliminates the effects of speculative execution on the HP machine. However, speculative execution is still evident on the Dell machine.

By recognising sequences of operations, an attacker can recover the bits of the exponent. Sequences of Square-Reduce-Multiply-Reduce indicate a set bit. Sequences of Square-Reduce which are not followed by Multiply indicate a clear bit. For example, in Fig. 6, between time slots 3,903 and 3,906 the calculated sequence is Square-Reduce, which is followed by a Square, indicating that in these time slots the victim was processing a clear bit.

Continuing throughout Fig. 6 we find that the bit sequence processed in this sample is 0110011010011. Table 1 shows the time slots corresponding to each bit.

Table 1: Time Slots for Bit Sequence

Seq.	Time Slots	Value	Seq.	Time Slots	Value
1	3,903–3,906	0	8	3,956–3,960	0
2	3,907–3,916	1	9	3,961–3,969	1
3	3,917–3,926	1	10	3,970–3,974	0
4	3,927–3,931	0	11	3,975–3,979	0
5	3,932–3,935	0	12	3,980–3,988	1
6	3,936–3,945	1	13	3,989–3,998	1
7	3,946–3,955	1			

System activity may cause the spy to miss time slots. The spy identifies missed time slot by noting jumps in the cycle counter. For example, In the run used for generating Fig. 6, the spy missed time slots 3,983 and 3,984. In this instance, the missed bits were not enough to hide the information on the bit processed during these time slots. However, if more slots are missed, data on bits of the private key exponent will be lost resulting in capture errors.

To measure the prevalence of capture errors, we used our spy program to observe and capture 1,000 signatures on each of the test configurations. We used a single invocation of a spy program to capture all the signatures in each system configuration. The GnuPG victim was executed from a shell in another window. Except for ensuring that the spy executes while running the signatures, the executions of the spy and of GnuPG are not synchronised.

For each observed signature, the spy outputs a text line representing the observed probes in each time slot. We used a shell script to parse this output and compared the results against the ground truth. The results are summarised in Table 2 and in Fig. 8. (For clarity, we trim Fig. 8 at 30% and 100 erroneous bits. A total of 15 samples have capture errors of more than 100 bits and the probability of no errors for the HP-CentOS configuration is 33%.)

Table 2: Statistics on Bit Errors in Capture

Hardware Software	HP Elite 8300		Dell PowerEdge T420	
	CentOS	VMware	CentOS	KVM
Average	1.41	26.55	25.12	66.12
Median	1	25	24	65
Max	15	196	96	190

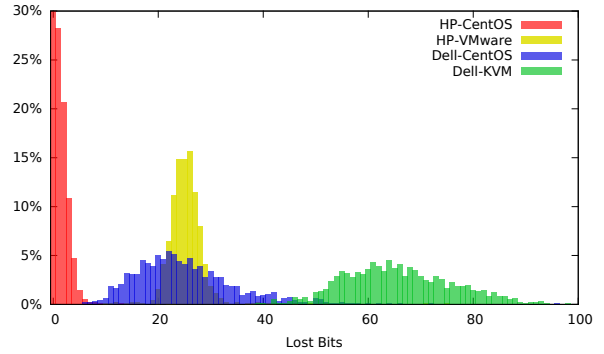


Figure 8: Distribution of Bit Errors in Capture

The shell script overestimates the number of errors. For example, due to the missing time slots, the script does not identify bit 12 in Table 1. We have manually inspected a few samples of capture output and estimate that manual inspection can reduce the number of errors by 25%-50%. Yet, the use of an automated script allows us to examine a large number of results.

On the HP machine we observe better results and significantly less noise than on the Dell machine. We believe this to be a consequence of the more advanced optimisations of the Xeon processor of the Dell machine. On each machine, results for the same-OS configuration are better than those for the cross-VM attack due to the added processing of the virtualisation layer.

Even accounting for the better results expected from manual inspection, the number of errors may be too big for a naïve brute force attack. Several strategies can be used to reduce the search space and to recover the private key. One such strategy is to rely on the nature of CRT-RSA exponentiation. As discussed in Section 2.3, an attacker only needs to recover one of the CRT components to break the encryption. By attacking the CRT component that has less errors, the attacker can reduce the search space to a more manageable size. Table 3 and Fig. 9 show the distribution of erroneous bits in the better captured CRT component in each signature. As these demonstrate, the search space is significantly reduced.

Several algorithms have been suggested for recovering the RSA exponent from partial information on the exponent bits [30, 31, 46]. These algorithms require between 27% and 70% of the bits of the exponent to recover the system key. While our attack reveals over 90% of the bits, it does not always recover the positions of

将探针移至更接近被测函数末端的缓存行，可消除HP机器上推测执行的影响。然而，Dell机器上仍能明显观察到推测执行现象。

攻击者通过识别操作序列，可推断出指数的比特位。当操作序列呈现平方约简-乘法约简-乘法约简模式时，表明存在设置位；若仅出现平方约简且未后续乘法操作，则提示存在明码位。例如图6所示，在3,903至3,906时间槽间，计算序列呈现平方约简模式，随后出现平方操作，说明受害者在此时段处理的是明码位。

如图6所示，该样本处理的比特序列为0110011010011。表1列出了各比特对应的时间槽。

表1：比特序列时隙

后迹	时间槽	值	后迹	时间槽	值
1	3,903-3,906	0	8	3,956-3,960	0
2	3,907-3,916	1	9	3,961-3,969	1
3	3,917-3,926	1	10	3,970-3,974	0
4	3,927-3,931	0	11	3,975-3,979	0
5	3,932-3,935	0	12	3,980-3,988	1
6	3,936-3,945	1	13	3,989-3,998	1
7	3,946-3,955	1			

系统运行时的活动可能导致监控程序漏检时隙。监控程序通过检测周期计数器的跳变来识别漏检时隙。例如，在生成图6的运行过程中，监控程序漏检了第3,983和3,984时隙。此时漏检的比特数不足以掩盖该时隙内处理的比特信息。但若漏检时隙过多，私钥指数比特的数据将丢失，从而导致捕获错误。

为测量捕获错误的发生率，我们使用间谍程序在每种测试配置下观察并捕获1,000个特征签名。通过单次调用间谍程序即可捕获所有系统配置中的特征签名。GnuPG攻击程序在另一窗口的shell中执行。除确保间谍程序在运行特征签名时执行外，间谍程序与GnuPG的执行过程并不同步。

针对每个观测到的特征信号，间谍程序会输出代表各时间槽观测探针的文本行。我们使用shell脚本解析该输出结果，并与真实数据进行比对。结果汇总于表2和图8中（为清晰起见，图8分别截取了30%和100个错误位的数据。共有15个样本的捕获错误超过100位，HP-CentOS配置的无错误概率为33%）。

表2：捕获位错误统计

硬件软件	惠普Elite 8300		戴尔PowerEdge T420	
	CentOS	虚拟机管理程序	CentOS	KVM
平均	1.41	26.55	25.12	66.12
中位数	1	25	24	65
最高的	15	196	96	190

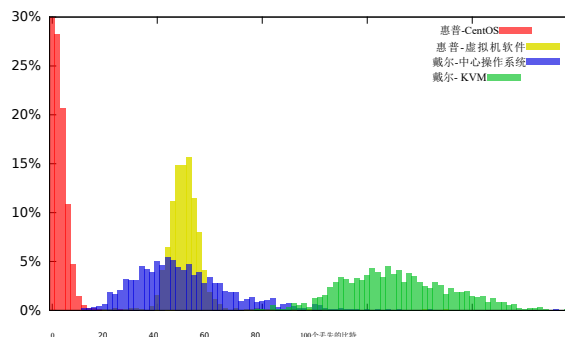


图8：捕获过程中比特错误的分布

该shell脚本对错误数量的估算存在高估现象。例如，由于时间槽缺失，脚本未能识别表1中的第12位。我们已人工检查了部分捕获输出样本，估计人工检查可使错误数量减少25%-50%。然而，使用自动化脚本可让我们对大量结果进行检查。

HP机器的测试结果更优，噪声水平也显著低于戴尔机器。我们认为这归因于戴尔机器搭载的至强处理器采用了更先进的优化技术。在相同操作系统配置下，由于虚拟化层的额外处理，两台机器的测试结果均优于跨虚拟机攻击的测试结果。

即便考虑到人工检查可能带来的更好结果，错误数量对于“ive暴力攻击”来说仍可能过大。为缩小搜索空间并恢复私钥，可采用多种策略。其中一种策略是利用CRT-RSA指数运算的特性。如第2.3节所述，攻击者只需恢复其中一个CRT组件即可破解加密。通过攻击错误较少的CRT组件，攻击者能将搜索空间缩减至更易管理的规模。表3和图9展示了各签名中捕获效果更佳CRT组件中错误位的分布情况。这些数据表明，搜索空间已得到显著压缩。

已有多算法被提出，用于从部分指数位信息中恢复RSA指数[30,31,46]。这些算法需要恢复系统密钥时，仅需27%至70%的指数位信息。虽然我们的攻击方法能揭示超过90%的位信息，但并不总能准确恢复位的位置。

Table 3: Statistics on Bit Errors in the Better Captured CRT Component

Hardware Software	HP Elite 8300		Dell PowerEdge T420	
	CentOS	VMware	CentOS	KVM
Average	0.20	11.75	7.11	28.66
Median	0	12	6	28
Max	4	68	26	47

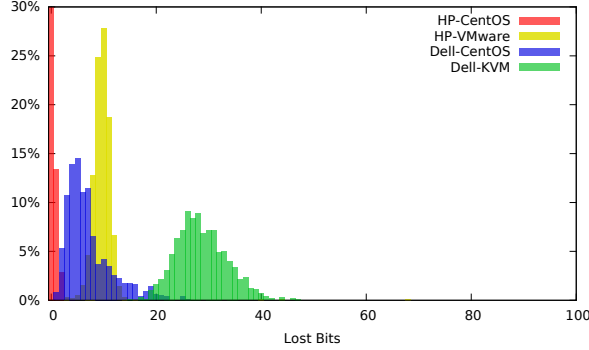


Figure 9: Distribution of Bit Errors in the Better Captured CRT Component

these bits. E.g. when a sequence of about 10 time slots is missed, this sequence can cover either one set bit or two clear bits. The attacker cannot, therefore, determine the bit positions of the following bits. Further research is required to determine whether these algorithms can be adapted to the data our attack recovers.

Another approach for recovering the key is to combine data from multiple signatures. As the positions of errors in each capture are independent, there is a small likelihood that any two captures will have errors in the same bit positions. To test this approach we manually merged the output of several pairs of observations of the spy under the Dell cross-VM scenario. When merging random pairs, we had at most a one bit error in the merged results. When merging the worst capture for the Dell cross-VM scenario with a random capture, the merged results had six bit errors, all of them in one of the CRT components and all have been identified during the process as potential errors. We, therefore, conclude that by observing two signatures, the attacker can recover the private key.

While the attack is very effective in recovering exponent bits, it does have some limitations. For the attack to work, the spy and the victim must execute on the same physical processor. For our testing, we set the processor affinity on the multi-processor system. However, in a real attack scenario the attack depends on the system scheduler.

When performing the tests, the spy and the victim were the only load on the system. Such a scenario is not representative of a real system where multiple processes are running. We expect such load to create noise that will

affect the quality of capture. Furthermore, for a load that includes multiple parallel instances of GnuPG, the spy will be unable to distinguish between memory access of each instance and will be unable to recover any data.

Another limitation is the length of the secret key. On the Dell machine, probing three memory locations takes about 2,200 cycles. Hence, the attack cannot work with time slots shorter than that. With shorter key lengths, time slots of 2,200 cycles or more do not provide enough resolution to trace the victim. Consequently, recovering the private key is more difficult with shorter keys, supporting the results of Walter [56].

5 Mitigation Techniques

The attack presented here is a real, immediate threat to computer security. It, therefore, raises the very pertinent question of countermeasures. The FLUSH+RELOAD attack relies on a combination of four factors for its operation: data flow from sensitive data to memory access patterns, memory sharing between the spy and the victim, accurate, high-resolution time measurements and the unfettered use of the `clflush` instruction. Preventing any of these blocks the attack.

The lack of permission checks for using the `clflush` instruction is a weakness of the X86 architecture. Consequently, the most complete solution to the problem is to limit the power of the `clflush` instruction. The main use of the `clflush` instruction is to enforce memory coherence, e.g. when using devices that do not support memory coherence [34]. Another potential use of the instruction is to control the use of the cache for improving program performance, e.g. by flushing lines that the program knows it will not require. However, we are not aware of any actual use of the instruction for this purpose.

As the first use is, clearly, a system function and the second is based on the assumption that no other process has access to the data, we suggest restricting the use of `clflush` to memory pages to which the process has write access and to memory pages to which the system allows `clflush` access. This access control could be implemented by adding memory types that restrict flush access to the PAT (Page Attribute Table) [35, chap. 11].

The ARM architecture [7] also includes instructions to evict cache lines. However, these instructions can only be used when the processor is in an elevated privilege mode. As such, the ARM architecture does not allow user process to selectively evict memory lines and the FLUSH+RELOAD is not applicable in this architecture.

Our attack seems not to work on contemporary AMD processors, such as the A10-6800K and Opteron 6348. The code in Fig. 5 returns the same result with and without the `clflush` instruction. Replacing the second

表3: 捕获更佳CRT组件的比特错误统计

硬件软件	惠普Elite 8300		戴尔PowerEdge T420	
	CentOS	虚拟机管理程序	CentOS	KVM
平均	0.20	11.75	7.11	28.66
中位数	0	12	6	28
最高的	4	68	26	47

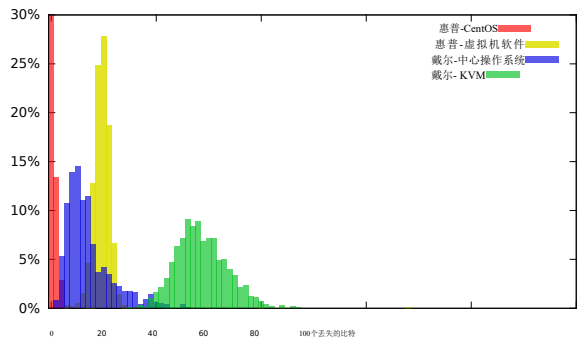


图9: 捕获更佳CRT组件的比特错误分布

这些比特位。例如，当约10个时隙的序列被遗漏时，该序列可能覆盖一个置位位或两个清零位。因此，攻击者无法确定后续比特位的位置。需要进一步研究以确定这些算法是否适用于我们攻击所恢复的数据。

另一种密钥恢复方法是整合多组特征数据。由于每次捕获的错误位置具有独立性，任意两组捕获数据在相同比特位出现错误的概率极低。为验证该方法，我们手动合并了戴尔交叉虚拟机场景下多组间谍活动观测数据。随机组合时，合并结果最多出现一位错误；而将戴尔交叉虚拟机场景中最差的捕获数据与随机数据合并后，结果出现六位错误，且全部位于CRT组件中，这些错误在处理过程中均被识别为潜在错误。因此我们得出结论：攻击者通过观察两组特征数据即可成功恢复私钥。

虽然该攻击在恢复指数位方面效果显著，但存在明显局限。攻击实施需依赖间谍程序与受害者运行在同一物理处理器上。在测试中，我们对多处理器系统设置了处理器亲和性参数。但在实际攻击场景中，其效果取决于系统调度器的配置。

在测试过程中，间谍程序与受害者是系统唯一的负载。此类场景无法代表实际运行多个进程的系统。我们预期此类负载会产生噪声，这将

影响捕获质量。此外，对于包含多个GnuPG并行实例的负载，间谍程序将无法区分各实例的内存访问，并且无法恢复任何数据。

另一个限制因素是密钥长度。在戴尔计算机上，探测三个内存地址需要约2,200个时钟周期。因此，攻击无法在更短的时间槽内实施。当密钥长度较短时，2,200个周期或更短的时间槽无法提供足够的分辨率来追踪受害者。因此，使用较短密钥时，恢复私钥的难度更大，这与Walter[56]的研究结果一致。

5 缓解技术

本文所述攻击对计算机安全构成真实且迫在眉睫的威胁，因此引发了关于应对措施的迫切问题。FLUSH+RELOAD攻击的运作依赖于四个要素的组合：敏感数据流向内存访问模式、间谍程序与受害者之间的内存共享、高精度时间测量以及clflush指令的无限制使用。若任一要素被阻断，攻击即告失效。

X86架构存在一个重大缺陷：在使用clflush指令时缺乏权限验证机制。因此，最彻底的解决方案就是限制该指令的权限范围。该指令的主要作用是强制执行内存一致性，例如在使用不支持内存一致性设备时[34]。另一个潜在用途是通过控制缓存使用来提升程序性能，比如清除程序已知不会使用的数据行。不过据我们所知，目前尚未发现该指令在这一方面的实际应用案例。

鉴于首次使用显然属于系统功能，而第二次使用基于其他进程无法访问数据的假设，我们建议将clflush的使用限制在进程具有写入权限的内存页以及系统允许clflush访问的内存页。这种访问控制可通过添加限制PAT（页面属性表）[35，第11章]中flush访问的内存类型来实现。

ARM架构[7]还包括清除缓存行的指令。然而，这些指令只能在处理器处于特权模式时使用。因此，ARM架构不允许用户进程选择性清除内存行，且FLUSH+RELOAD不适用于该架构。

我们的攻击方法似乎对当代AMD处理器（如A10-6800K和Opteron 6348）无效。图5中的代码无论是否使用clflush指令均返回相同结果。替换第二个

`rdtsc` instruction (Line 12) with the similar `rdtscp` instruction fixes this issue, however, two problems prevent the use of the technique. The first problem is that data seems to linger in the cache for some time after being evicted. The second problem is that the attack does not capture accesses from other processes. A possible explanation for this behaviour is that the AMD caches are non-inclusive, i.e. data in L1 does not need to also be in L2 or L3, as is the case with the Intel caches. Consequently, evicting data from the LLC does not, necessarily, evicts it from the L1 caches of other cores. Processes executing on other cores can access data in the L1 cache without triggering a load from memory to the LLC. The attack does work on older AMD processors, such as the Opteron 2212.

Hardware based countermeasures, such as those described above cannot provide an immediate solution to the problem. They will take time to develop and will not protect existing hardware. Consequently, for immediate mitigation of the attack, software-based solutions are required.

Another possible solution is preventing sharing between the spy and the victim. Preventing page sharing between processes provides protection against the FLUSH+RELOAD attack. However, this approach goes against the trend of increased sharing in operating systems and virtualisation hypervisors. Completely eliminating page sharing would significantly increase the memory requirements of modern operating systems and is, therefore, unlikely to be a feasible solution. As a partial solution, it may be possible to avoid sharing of sensitive code by changing the program loader. Another partial solution is disabling page de-duplication, which prevents using the FLUSH+RELOAD attack between co-hosted guests in a virtualised system. This approach is recommended for public compute clouds which offer the implied promise that guests cannot interfere with each other.

Software diversification [24] is a collection of techniques that permute the locations of objects within the address spaces of processes. While most of these techniques were originally developed as a protection against memory corruption attacks, some of them can be used to prevent sharing and, consequently, to mitigate the FLUSH+RELOAD attack. More specifically, in virtualised environments, static reordering of code and data [12, 24, 36] can be used to create unique copies of programs in each virtual machines. As these copies are not available outside the specific virtual machine, pages of the program are not de-duplicated and sharing is prevented. Diversifying the program at run time [22] can prevent sharing of the program text even when the attacker has access to the binary file. As discussed above, the FLUSH+RELOAD technique is oblivious to the virtual to physical address

mapping. Consequently, diversification techniques that rely on permuting the virtual address mapping of code pages, such as [50, 59], do not provide any protection against the attack.

FLUSH+RELOAD, like other side-channel attacks, relies on the availability of a high-resolution clock. Reducing the resolution of the clock or introducing noise to clock measurement [32, 53] can be used as a countermeasure against the attack. The main limitation of this approach is that the attacker can use other methods for generating high resolution clocks. Examples include using data from the network or running a ‘clock’ process in a separate execution core.

Irrespective of the measures described above, cryptographic software should be protected against the attack. Following our disclosure [18, 38], the GnuPG team released GnuPG version 1.4.14 and libgcrypt version 1.5.3. These mitigate the attack using the square-and-multiply-always [21] algorithm, shown in Listing 10. The algorithm executes the square and the multiply steps for each bit, but ignores the result of the multiply step for bits of value 0.

```
function exponent( $b, e, m$ )
begin
   $x \leftarrow 1$ 
  for  $i \leftarrow |e| - 1$  downto 0 do
     $x \leftarrow x^2$ 
     $x \leftarrow x \bmod m$ 
     $x' \leftarrow xb$ 
     $x' \leftarrow x' \bmod m$ 
    if ( $e_i = 1$ ) then
       $x = x'$ 
    endif
  done
  return  $x$ 
end
```

Figure 10: Exponentiation by Square-and-Multiply-Always

When introducing instructions with no effect, care should be taken to prevent the compiler from optimising these away. In the case of the GnuPG fix, the optimiser cannot know that the added addition does not have side-effects. With the possibility of side-effects, the optimiser takes a conservative approach and invokes the function.

The implementation still contains a small section of code that depends on the value of the bit, which could, theoretically, be exploited by a cache side-channel attack. However, due to speculative execution, the processor is likely to access the section irrespective of the value of the bit. Furthermore, as this section is short and is smaller than a cache line, it is likely to fit within the same cache line as the preceding or following code. Hence, we believe that this implementation protects against the FLUSH+RELOAD attack.

使用与rdtscp指令类似的rdtsc指令（第12行）可以解决这个问题，但存在两个问题阻碍该技术的应用。第一个问题是数据在被驱逐后似乎会在缓存中滞留一段时间。第二个问题是该攻击无法捕获其他进程的访问行为。这种行为的可能解释是AMD缓存具有非包容性，即L1缓存中的数据无需像英特尔缓存那样同时存在于L2或L3缓存中。因此，从LLC（乱序缓存）中驱逐数据并不必然导致其他核心的L1缓存中数据被驱逐。运行在其他核心上的进程可以访问L1缓存中的数据，而无需触发内存到LLC的加载操作。该攻击确实适用于较旧的AMD处理器，例如Opteron 2212。

基于硬件的防御措施（如前述方案）无法立即解决问题，其开发需要时间且无法保护现有硬件。因此，为快速缓解攻击，必须采用基于软件的解决方案。

另一种可行方案是阻止间谍程序与受害者之间的共享。通过禁止进程间的页面共享，可有效防范F LUSH+R ELOAD攻击。但这种方法与操作系统及虚拟化管理程序中日益增长的共享趋势背道而驰。若完全消除页面共享，将大幅增加现代操作系统的内存需求，因此难以实现。作为折中方案，可通过修改程序加载器来避免敏感代码共享。另一种方法是禁用页面去重功能，该功能可防止虚拟化系统中托管的客户机之间使用F LUSH+R ELOAD攻击。此方案特别适用于公共计算云环境，这类云服务默认承诺客户机之间不会相互干扰。

软件多样化[24]是一组通过置换进程地址空间中对象位置的技术集合。虽然这些技术最初多为防范内存破坏攻击而开发，但其中部分可用来防止共享，从而缓解F LUSH+R ELOAD攻击。具体而言，在虚拟化环境中，通过代码和数据的静态重排[12,24,36]可在每个虚拟机中创建程序的唯一副本。由于这些副本仅存在于特定虚拟机内部，程序页面不会被去重且共享被阻止。运行时的程序多样化[22]即使攻击者能访问二进制文件，也能防止程序文本的共享。如前所述，F LUSH+R ELOAD技术对虚拟地址与物理地址的映射关系无感知。

映射。因此，依赖于代码页虚拟地址映射置换的多样化技术（如[50,59]）无法提供任何针对该攻击的防护。

F LUSH+R ELOAD与其他侧信道攻击类似，其实施依赖高分辨率时钟的可用性。降低时钟分辨率或在时钟测量中引入噪声[32,53]可作为对抗该攻击的措施。该方法的主要局限在于攻击者可能采用其他生成高分辨率时钟的方法，例如利用网络数据或在独立执行核心中运行‘时钟’进程。

无论采取何种防护措施，加密软件都必须具备抵御攻击的能力。根据我们此前的披露[18,38]，GnuPG团队发布了1.4.14版本的GnuPG和1.5.3版本的libgcrypt。这些版本通过采用始终平方与乘法[21]算法（如清单10所示）来缓解攻击。该算法对每个比特位执行平方和乘法操作，但对于值为0的比特位则忽略乘法运算结果。

```

函数指数 (b, e, m)
开始
x ← 1
对于 i ← jej - 1 从 0 到 x ←
  x2
  x ← x mod m
  x' ← xb
  x' ← x' 模 m
  如果 (ei =
    1) 则 x = x'
  如果
已返回x
端

```

图10：平方乘法的指数运算

在引入无效指令时，需注意防止编译器对其进行优化。以GnuPG修复方案为例，优化器无法确认新增加法操作不会产生副作用。考虑到潜在副作用，优化器采取保守策略并调用该函数。

该实现方案仍包含一小段依赖于位值的代码，理论上可能被缓存侧信道攻击利用。但由于采用推测执行机制，处理器很可能无论位值如何都会访问该代码段。此外，由于这段代码较短且小于缓存行大小，很可能与前后代码共用同一缓存行。因此我们认为，该实现方案能够有效抵御F LUSH+R ELOAD攻击。

This fix, however, does not protect against other forms of side-channel attack. In particular, the code is likely to be vulnerable to Branch Prediction Analysis [3]. Furthermore, as access patterns to data depend on the values of the exponent bits, the code is likely to be vulnerable to PRIME+PROBE attacks [51,61]. Like FLUSH+RELOAD, these side-channel attacks rely on data flow from secret exponent bits to memory access patterns. These attacks can be prevented by using *constant time* exponentiation, where the sequence of instructions and memory locations accessed are fixed and do not depend on the value of the exponent bits. Techniques for constant time computation have been explored in the NaCl cryptographic library [10]. The pattern of accesses to memory lines of the OpenSSL [41] implementation of RSA exponentiation is not dependent on secret exponent bits. Consequently, even though the implementation is not constant time [11], it is not vulnerable to our attack.

Constant time computation is not, however, a panacea for the problem of side-channel attacks. FLUSH+RELOAD can be applied to extract secret data from non cryptographic software. For such software, the performance costs of constant-time computation are unreasonable, hence other solutions are required.

6 Related Work

Several works have pointed out that page sharing exposes guests to information leakage, which can be exploited for implementing covert channels [58], OS fingerprinting [44] and for detecting applications and data in other guests [49]. These works exploit the copy-on-write feature of page sharing. Copy-on-write introduces a significant delay when a page is copied. Hence, by timing write operations on pages, a spy can deduce the existence of pages with identical contents in other guests. As page de-duplication is a slow process, all these attacks have a very low resolution.

Using a cache side-channel to trace the execution of a program is not a new idea [1, 2, 4, 14, 19, 29, 61]. In all of these attacks, the victim and the spy must share the execution core, either by using hyper-threading or by interleaving the execution of the victim and the spy on the same core.

Gullasch et al. [29] describes an attack on AES which traces the victim's access to the S-Boxes. Our work builds on the attack technique presented by Gullasch et al. and extends it in two ways. Gullasch et al. only applies the attack on a time-shared core and does not exploit the eviction from a shared LLC. Our attack exposes the use of a shared LLC and demonstrates that the technique can be used across cores. Additionally, Gullasch et al. uses the `cpuid` instruction to synchronise the instruction stream whereas we use fence instructions. In

virtualised environments, the `cpuid` is emulated in software and this emulation takes over 1,000 cycles. With two `cpuid` instructions in each probe, the Gullasch et al. probe spans over 2,500 cycles. As our attack requires three probes within 2,500 cycles, the resolution of the Gullasch et al. code is not high enough for implementing our cross-VM attack.

The attack in Zhang et al. [61] specifically targets virtualised environments, extracting the private ElGamal [23] key of a GnuPG decryption executing in another guest. The attack depends on a weakness in the scheduler of the Xen hypervisor [8]. The granularity of the attack is one probe in 50,000 cycles, limiting the minimum size of victim key that can be captured. The modulus in the paper is 4,096 bits long. The attack has low signal to noise ratio, and requires the use of filtering. Even with this filtering and the large modulus, the attack requires six hours of constant decryption to recover the key.

Weiß et al. [57] also describes cache timing attack in a virtualised environment. The attack is an adaptation of Bernstein's attack [9] that relies on the short constant communication time between domains in the L4 kernel.

7 Conclusions

In this paper we describe the FLUSH+RELOAD technique and how we use it to extract GnuPG private keys across multiple processor cores and across virtual machine boundaries.

It is hard to overstate the severity of the attack, both in virtualised and in non-virtualised environments. GnuPG is a very popular cryptographic package. It is used as the cryptography module of many open-source projects and is used, for example, for email, file and communication encryption. Hence, vulnerable versions of GnuPG are not safe for multi-tenant systems or for any system that may run untrusted code.

While significant, the attack on GnuPG is only a demonstration of the power of the FLUSH+RELOAD technique. The technique is generic and can be used to monitor other software. It can be used to devise other types of attacks on cryptographic software. It can also be used against other types of software. For example, it could be used to collect statistical data on network traffic by monitoring network handling code or it could monitor keyboard drivers to collect keystroke timing information.

Hence, while the GnuPG team has fixed the vulnerability in their software, their fix does not address the broader threat exposed by this paper.

The FLUSH+RELOAD technique exploits the lack of restrictions on the use of the `clflush` instruction. Not restricting the use of the instruction is a security weakness of the Intel implementation of the X86 architecture. This enables processes to interact using read-only pages.

然而,这一修复并不能防范其他形式的侧信道攻击。特别是,代码很可能容易受到分支预测分析[3]的攻击。此外,由于数据访问模式取决于指数位的值,代码很可能容易受到PRIME+PROBE攻击[51,61]的攻击。与FLUSH+RELOAD类似,这些侧信道攻击依赖于从秘密指数位到内存访问模式的数据流。这些攻击可以通过使用常数时间指数运算来防止,其中指令序列和访问的内存位置是固定的,不依赖于指数位的值。常数时间计算技术已在氯化钠加密库[10]中进行了探索。OpenSSL[41]实现的RSA指数运算对内存行的访问模式不依赖于秘密指数位。因此,尽管该实现不是常数时间[11],但它不会受到我们的攻击。

然而,常数时间计算并非解决侧信道攻击问题的万能药。FLUSH+RELOAD无法从非加密软件中提取秘密数据。对于此类软件,常数时间计算的性价比过高,因此需要其他解决方案。

6 相关工作

多项研究指出,页共享机制会使访客面临信息泄露风险,这些信息可能被用于构建隐蔽通道[58]、进行操作系统指纹识别[44],以及检测其他访客中的应用程序和数据[49]。这些研究利用了页共享的写时复制特性。写时复制会在页面复制时引入显著延迟,因此通过精确控制页面写入操作的时机,间谍程序能够推断出其他访客中存在内容完全相同的页面。由于页面去重过程耗时较长,所有这些攻击手段的侦测精度都相当有限。

利用缓存侧信道追踪程序执行并非新思路[1,2,4,14,19,29,61]。在所有这些攻击中,受害者与间谍必须共享执行核心,可通过超线程技术实现,或通过相同核心上交错执行受害者与间谍来达成。

古拉斯赫等人[29]提出了一种针对AES加密算法的攻击方法,该方法通过追踪受害者对S盒的访问行为来实施攻击。我们的研究在古拉斯赫团队提出的攻击技术基础上,从两个维度进行了拓展。古拉斯赫团队仅针对时间共享核心实施攻击,未利用共享LLC的驱逐机制。我们的攻击不仅暴露了共享LLC的使用场景,还证明该技术可跨核心通用。此外,古拉斯赫团队采用cpuid指令来同步指令流,而我们则使用了fence指令。

在虚拟化环境中,CPUID通过软件模拟实现,该模拟过程耗时超过1,000个时钟周期。由于每个探测器包含两条CPUID指令,Gullasch等人设计的探测器可覆盖超过2,500个时钟周期。鉴于我们的攻击需要在2,500个时钟周期内完成三次探测,Gullasch等人代码的分辨率不足以支持我们跨虚拟机攻击的实施。

张等人[61]提出的攻击方法专门针对虚拟化环境,通过提取在其他客户机上运行的GnuPG解密程序的私有ElGamal[23]密钥。该攻击利用了Xen虚拟机监控程序[8]调度器的漏洞。攻击的粒度设置为每5万次运行周期检测一次,这限制了可捕获的最小密钥尺寸。论文中使用的模数为4096位。由于信噪比较低,攻击需要借助过滤技术。即便采用过滤措施和大模数特性,攻击仍需持续六小时的不间断解密才能成功获取密钥。

Weiß等人[57]还描述了虚拟化环境中的缓存定时攻击。该攻击是伯恩斯坦攻击[9]的改进版本,其原理基于L4内核中域间短暂的恒定通信时间。

7 结论

本文描述了FLUSH+RELOAD技术,以及如何利用该技术跨多个处理器核心和虚拟机边界提取GnuPG私钥。

无论在虚拟化还是非虚拟化环境中,这次攻击的严重性都难以言表。GnuPG作为广受欢迎的加密工具包,不仅是众多开源项目的加密模块,还广泛应用于电子邮件、文件及通信加密等领域。因此,GnuPG的漏洞版本对多租户系统或任何可能运行不可信代码的系统而言,都存在安全隐患。

虽然攻击GnuPG的事件意义重大,但它仅是FLUSH+RELOAD技术威力的展示。该技术具有通用性,可用于监控其他软件,还能设计针对加密软件的各类攻击方案,甚至适用于其他类型软件。例如,通过监控网络处理代码可收集网络流量统计信息,或通过监控键盘驱动程序获取按键时序数据。

因此,尽管GnuPG团队已修复其软件中的漏洞,但该修复方案并未解决本文所揭示的更广泛威胁。

FLUSH+RELOAD技术利用了clflush指令使用不受限制的特性。这种指令使用不受限制是Intel X86架构实现中的安全漏洞,使得进程能够通过只读页进行交互。

Addressing this weakness requires a hardware fix, which, unless implemented as a microcode update, will not be applicable to existing hardware.

Preventing page sharing also blocks the FLUSH+RELOAD technique. Given the strength of the attack, we believe that the memory saved by sharing pages in a virtualised environment does not justify the breach in the isolation between guests. We, therefore, recommend that memory de-duplication be switched off.

Acknowledgments

We would like to thank the anonymous reviewers and our shepherd, Thomas Ristenpart, for their valuable comments and support.

This research was performed under contract to the Defence Science and Technology Organisation (DSTO) Maritime Division, Australia.

References

- [1] ACHIÇMEZ, O. Yet another microarchitectural attack: exploiting I-Cache. In *Proceedings of the ACM Workshop on Computer Security Architecture* (Fairfax, Virginia, United States, November 2007), P. Ning and V. Atluri, Eds., pp. 11–18.
- [2] ACHIÇMEZ, O., BRUMLEY, B. B., AND GRABHER, P. New results on instruction cache attacks. In *Proceedings of the Workshop on Cryptographic Hardware and Embedded Systems* (Santa Barbara, California, United States, April 2010), S. Mangard and F.-X. Standaert, Eds., pp. 110–124.
- [3] ACHIÇMEZ, O., KOÇ, Ç. K., AND SEIFERT, J.-P. On the power of simple branch prediction analysis. In *Proceedings of the Second ACM Symposium on Information, Computer and Communication Security* (Singapore, March 2007), pp. 312–320.
- [4] ACHIÇMEZ, O., AND SCHINDLER, W. A vulnerability in RSA implementations due to instruction cache analysis and its demonstration on OpenSSL. In *Proceedings of the Cryptographers' Track at the RSA Conference* (San Francisco, California, United States, April 2008), T. Malkin, Ed., pp. 256–273.
- [5] ACHIÇMEZ, O., SCHINDLER, W., AND KOÇ, Ç. K. Cache based remote timing attacks on the AES. In *Proceedings of the Cryptographers' Track at the RSA Conference* (San Francisco, California, United States, February 2007), M. Abe, Ed., pp. 271–286.
- [6] ARCANGELI, A., EIDUS, I., AND WRIGHT, C. Increasing memory density by using KSM. In *Proceedings of the Linux Symposium* (Montreal, Quebec, Canada, July 2009), pp. 19–28.
- [7] *ARM Architecture Reference Manual*, ARMv7-A and ARMv7-R ed., 2012.
- [8] BARHAM, P., DRAGOVIC, B., FRASER, K., HAND, S., HARRIS, T., HO, A., NEUGEBAUER, R., PRATT, I., AND WARFIELD, A. Xen and the art of virtualization. In *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles* (Bolton Landing, New York, United States, October 2003), M. L. Scott and L. L. Peterson, Eds., ACM, pp. 164–177.
- [9] BERNSTEIN, D. J. Cache-timing attacks on AES. <http://cr.yp.to/antiforgery/cachetiming-20050414.pdf>, April 2005.
- [10] BERNSTEIN, D. J., LANGE, T., AND SCHWABE, P. The security impact of a new cryptographic library. In *Proceedings of the Second International Conference on Cryptology and Information Security in Latin America* (Santiago, Chile, October 2012), A. Hevia and G. Neven, Eds., pp. 159–176.
- [11] BERNSTEIN, D. J., AND SCHWABE, P. A word of warning. CHES 2013 Rump Session, August 2013.
- [12] BHATKAR, S., DUVARNEY, D. C., AND SEKAR, R. Address obfuscation: an efficient approach to combat a broad range of memory error exploits. In *Proceedings of the USENIX Security Symposium* (Washington, DC, United States, August 2003), pp. 105–120.
- [13] BOBROW, D. G., BURCHFIELD, J. D., MURPHY, D. L., AND TOMLINSON, R. S. TENEX, a paged time sharing system for the PDP-10. *Communications of the ACM* 5, 3 (March 1972), 135–143.
- [14] BRUMLEY, B. B., AND HAKALA, R. M. Cache-timing template attacks. In *Advances in Cryptology - ASIACRYPT 2009* (2009), M. Matsui, Ed., vol. 5912 of *Lecture Notes in Computer Science*, Springer-Verlag, pp. 667–684.
- [15] BUGNION, E., DEVINE, S., GOVIL, K., AND ROSENBLUM, M. Disco: Running commodity operating systems on scalable multiprocessors. *ACM Transactions on Computer Systems* 15, 4 (November 1997), 412–447.
- [16] CAMPAGNA, M., AND SETHI, A. Key recovery method for CRT implementation of RSA. Report 2004/147, IACR Cryptology ePrint Archive, 2004.
- [17] CERON, R., FOLCO, R., LEITAO, B., AND TSUBAMOTO, H. *Power Systems Memory Deduplication*. IBM, September 2012.
- [18] CERT vulnerability note vu#976534: L3 cpu shared cache architecture is susceptible to a Flush+Reload side-channel attack. <http://www.kb.cert.org/vuls/id/976534>, October 2013.
- [19] CHEN, C., WANG, T., KOU, Y., CHEN, X., AND LI, X. Improvement of trace-driven I-Cache timing attack on the RSA algorithm. *The Journal of Systems and Software* 86, 1 (2013), 100–107.
- [20] CIPRESSO, T., AND STAMP, M. Software reverse engineering. In *Handbook of Information and Communication Security*, P. Stavroulakis and M. Stamp, Eds. Springer, 2010, ch. 31, pp. 659–696.
- [21] CORON, J.-S. Resistance against differential power analysis for elliptic curve cryptosystems. In *Proceedings of the Workshop on Cryptographic Hardware and Embedded Systems* (Worcester, Massachusetts, United States, August 1999), Ç. K. Koç and C. Paar, Eds., pp. 292–302.
- [22] CURTSINGER, C., AND BERGER, E. D. STABILIZER: Statistically sound performance evaluation. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems* (Houston, Texas, United States, March 2013), pp. 219–228.
- [23] ELGAMAL, T. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory* 31, 4 (July 1985), 469–472.
- [24] FORREST, S., SOMAYAJI, A., AND ACKLEY, D. H. Building diverse computer systems. In *Proceedings of the Sixth Workshop on Hot Topics in Operating Systems* (Cape Code, Massachusetts, United States, May 1997), pp. 67–72.
- [25] GARNER, H. L. The residue number system. *IRE Transactions on Electronic Computers EC-8*, 2 (June 1959), 140–147.
- [26] GINGELL, R. A., LEE, M., DANG, X. T., AND WEEKS, M. S. Shared libraries in SunOS. In *USENIX Conference Proceedings* (Phoenix, Arizona, United States, Summer 1987), pp. 131–145.

要解决这一缺陷, 需要进行硬件修复, 但若采用微代码更新的方式, 该修复将无法适用于现有硬件。

防止页面共享还会阻止FLUSH+RELOAD技术。鉴于攻击的强度, 我们认为在虚拟化环境中共享页面所节省的内存并不能证明破坏客户机隔离的合理性。因此, 我们建议关闭内存去重。

致谢

我们谨此感谢匿名审稿人及我们的导师Thomas Ristenpart提供的宝贵意见与支持。

本研究系根据澳大利亚国防科技组织(DSTO)海事部的合同开展。

参考文献

- [1] ACIİÇMEZ, O. 另一种微架构攻击: 利用I-Cache. 收录于 *ACM 计算机安全架构研讨会论文集* (美国弗吉尼亚州费尔法克斯, 2007年11月), P. Ning与V. Atluri编, 第11-18页。
- [2] ACIİÇMEZ, O., B RUMLEY, B.B., 与 G RABHER, P. 关于指令缓存攻击的新研究结果. 收录于 *加密硬件与嵌入式系统研讨会论文集* (美国加利福尼亚州圣巴巴拉, 2010年4月), S. Mangard与F.-X. Standaert编, 第110-124页。
- [3] ACIİÇMEZ, O., KOÇ, C. K., 和 S EIFERT, J.-P. 简单分支预测分析的威力. 收录于 *第二届ACM信息、计算机与通信安全研讨会论文集* (新加坡, 2007年3月), 第312-320页。
- [4] ACIİÇMEZ, O., 与 S chindler, W. 由于指令缓存分析导致的RSA实现漏洞及其在OpenSSL上的演示. 发表于 *RSA会议密码学家分会论文集* (美国加利福尼亚州旧金山, 2008年4月), T. Malkin编, 第256-273页。
- [5] ACIİÇMEZ, S·金德勒, W·与 K·OÇ·K·Cache基于缓存的AES远程定时攻击. 收录于 *《RSA会议密码学家分会论文集》(2007年2月, 美国加利福尼亚州旧金山)*, 由M·Abe主编, 第271-286页。
- [6] Arcangel, A., E IDUS, I., AND W right, C. 通过使用KSM提高存储密度. 收录于 *Linux研讨会论文集* (加拿大魁北克省蒙特利尔, 2009年7月), 第19-28页。
- [7] *ARM架构参考手册*, ARMv7-A和ARMv7-R版, 2012年。
- [8] BARHAM, P., Dragovic, B., Fraser, K., H 与 S., Harris, T., H O, A., N eugebauer, R., P RATT, I., 与 W arfield, A. Xen与虚拟化艺术. 见 *第十九届ACM操作系统原理研讨会论文集* (美国纽约州博尔顿兰丁, 2003年10月), M. L. Scott与L. L. Peterson编, ACM出版社, 第164-177页。
- [9] Bernstein, D.J. AES的缓存定时攻击. <http://cr.yp.to/antiforgery/cachetiming-20050414.pdf>, 2005年4月。
- [10] Bernstein, D. J., Lange, T., 与 Schwabe, P. 新加密库的安全影响. 见 *第二届拉丁美洲密码学与信息安全国际会议论文集* (智利圣地亚哥, 2012年10月), A. Hevia 与 G. Neven 主编, 第159-176页。
- [11] Bernstein, D.J., 与 Schwabe, P. 《一个警告》. CHES 2013年8月Rump会议。
- [12] Bhattacharjee, S.D. U V ARNEY, D.C., 与 Sekar R. 地址混淆: 一种有效对抗多种内存错误攻击的方案. 收录于 *USENIX 安全研讨会论文集* (美国华盛顿特区, 2003年8月), 第105-120页。
- [13] Bobrow, D. G., Burchfiel, J. D., Murphy, D. L., 与 Tomlinson, R. S. TENEX, 一种用于PDP-10的分页分时系统. *ACM通讯* 5, 3 (1972年3月), 135-143。
- [14] B RUMLEY, B.B. 与 Hakala, R.M. 缓存定时模板攻击. 收录于 *密码学进展-asiacrypt 2009* (2009), M. Matsui编, *计算机科学讲义* 第5912卷, Springer-Verlag出版社, 第667-684页。
- [15] Bugnion, E., Devine, S., G ovil, K., 与 Rosenblum, M. Disco: 在可扩展多处理器上运行通用操作系统. *ACM 计算机系统汇刊* 15, 4 (1997年11月), 412-447。
- [16] Campagna, M., 与 Sethi, A. RSA的CRT实现密钥恢复方法. 报告2004/147, IACR 密码学电子预印本档案, 2004年。
- [17] Ceron, R., Folco, R., Leita, B., AND tsubamoto, H. 电力系统中内存去重. IBM, 2012年9月。
- [18] CERT漏洞公告vu#976534: L3 CPU共享缓存架构存在Flush+Reload侧信道攻击风险. <http://www.kb.cert.org/vuls/id/976534>, 2013年10月。
- [19] Chen, C., Wang, T., Kou, Y., Chen, X., AND Li, X. 改进基于跟踪驱动的I-Cache时序攻击对RSA算法的攻击. *系统与软件杂志* 86, 1 (2013), 100-107。
- [20] Cipresso, T., 和 Stamp, M. 软件逆向工程. 见 *《信息与通信安全手册》*, P. Stavroulakis 和 M. Stamp 编. 斯普林格出版社, 2010年, 第31章, 第659-696页。
- [21] Coron, J.-S. 椭圆曲线密码系统对差分功耗分析的抗性. 收录于 *密码硬件与嵌入式系统研讨会论文集* (美国马萨诸塞州伍斯特市, 1999年8月), C·K·科克与C·帕尔编, 第292-302页。
- [22] Curtsinger, C. 和 Berger, E. D. 稳定器: 统计学上合理的性能评估. 见 *第18届国际编程语言和操作系统架构支持会议论文集* (美国德克萨斯州休斯顿, 2013年3月), 第219-228页。
- [23] ElGamal, T. 基于离散对数的公钥密码系统与签名方案. *IEEE信息论汇刊* 31卷, 第4期 (1985年7月), 469-472页。
- [24] Forrest, S., omayaj, A., 与 Ackley, D.H. 构建多样化的计算机系统. 见 *第六届操作系统热点专题研讨会论文集* (美国马萨诸塞州开普科德, 1997年5月), 第67-72页。
- [25] Garner, H. L. 残基编号系统. *IRE电子计算机汇刊EC-8*, 2 (1959年6月), 140-147。
- [26] Ingell, R. A., Lee, M., Dang, X. T., AND Weeks, M. S. SunOS中的共享库. 见 *USENIX 会议论文集* (美国亚利桑那州凤凰城, 1987年夏季), 第131-145页。

- [27] GNU Privacy Guard. <http://www.gnupg.org>, 2013.
- [28] GORDON, D. M. A survey of fast exponentiation methods. *Journal of Algorithms* 27, 1 (April 1998), 129–146.
- [29] GULLASCH, D., BANGERTER, E., AND KRENN, S. Cache games — bringing access-based cache attacks on AES to practice. In *Proceedings of the IEEE Symposium on Security and Privacy* (Oakland, California, United States, may 2011), pp. 490–595.
- [30] HENINGER, N., AND SHACHAM, H. Reconstructing RSA private keys from random key bits. In *Proceedings of the 29th Annual International Cryptology Conference (CRYPTO 2009)* (Santa Barbara, California, United States, August 2009), S. Halevi, Ed., pp. 1–17.
- [31] HERMANN, M., AND MAY, A. Solving linear equations modulo divisors: On factoring given any bits. In *Advances in Cryptology - ASIACRYPT 2008* (Melbourne, Australia, December 2008), vol. 5350 of *Lecture Notes in Computer Science*, pp. 406–424.
- [32] HU, W.-M. Reducing timing channels with fuzzy time. In *Proceedings of the IEEE Symposium on Security and Privacy* (Oakland, California, United States, May 1991), pp. 8–20.
- [33] HUFFMAN, C. Memory combining in Windows 8 and Windows Server 2012. <http://blogs.technet.com/b/clint/archive/2012/11/29/memory-combining-in-windows-8-and-windows-server-2012.aspx>, November 2012.
- [34] INTEL CORPORATION. *Intel 64 and IA-32 Architecture Optimization Reference Manual*, April 2012.
- [35] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developer's Manual Volume 3A: System Programming Guide, Part 1*, March 2013.
- [36] KIL, C., JUN, J., BOOKHOLT, C., XU, J., AND NING, P. Address space layout permutation (aslp): Towards fine-grained randomization of commodity software. In *Proceedings of the Annual Computer Security Applications Conference* (Miami Beach, Florida, United States, December 2006), pp. 339–348.
- [37] KIVITY, A., KAMAY, Y., LAOR, D., LUBLIN, U., AND LIGUORI, A. *kvm*: the Linux virtual machine monitor. In *Proceedings of the Linux Symposium* (Ottawa, Ontario, Canada, June 2007), vol. one, pp. 225–230.
- [38] KOCH, W. GnuPG 1.4.14 released. <http://lists.gnupg.org/pipermail/gnupg-announce/2013q3/000330.html>, July 2013.
- [39] MIŁOŚ, G., MURRAY, D. G., HAND, S., AND FETTERMAN, M. A. Satori: Enlightened page sharing. In *Proceedings of the 2009 USENIX Annual Technical Conference* (San Diego, California, United States, June 2009).
- [40] MURPHY, D. L. Storage organization and management in TENEX. In *Proceedings of the Fall Joint Computer Conference, AFIPS'72, Part I* (Anaheim, California, United States, December 1972), pp. 23–32.
- [41] OPENSSL. <http://www.openssl.org>.
- [42] ORGANICK, E. I. *The Multics System: An Examination of Its Structure*. The MIT Press, 1972.
- [43] OSVIK, D. A., SHAMIR, A., AND TROMER, E. Cache attacks and countermeasures: the case of AES. <http://www.cs.tau.ac.il/~tromer/papers/cache.pdf>, November 2005.
- [44] OWENS, R., AND WANG, W. Non-interactive OS fingerprinting through memory de-duplication technique in virtual machines. In *Proceedings of the 30th IEEE International Performance Computing and Communications Conference* (Orlando, Florida, United States, November 2011), S. Zhong, D. Dou, and Y. Wang, Eds., IEEE, pp. 1–8.
- [45] PAOLONI, G. *How to Benchmark Code Execution Times on Intel IA-32 and IA-64 Instruction Set Architectures*. Intel Corporation, September 2010.
- [46] PERCIVAL, C. Cache missing for fun and profit. <http://www.daemonology.net/papers/htt.pdf>, 2005.
- [47] RISTENPART, T., TROMER, E., SHACHAM, H., AND SAVAGE, S. Hey, you, get off my cloud: Exploring information leakage in third-party compute clouds. In *Proceedings of the 16th ACM Conference on Computer and Communication Security* (Chicago, Illinois, United States, November 2009), E. Al-Shaer, S. Jha, and A. D. Keromytis, Eds., pp. 199–212.
- [48] RIVEST, R. L., SHAMIR, A., AND ADLEMAN, L. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM* 21, 2 (February 1978), 120–126.
- [49] SUZAKI, K., IJIMA, K., YAGI, T., AND ARTHO, C. Memory deduplication as a threat to the guest. In *Proceedings of the 2011 European Workshop on System Security* (Salzburg, Austria, 2011).
- [50] The PaX project. <http://pax.grsecurity.net/>.
- [51] TROMER, E., OSVIK, D. A., AND SHAMIR, A. Efficient cache attacks in AES, and countermeasures. *Journal of Cryptology* 23, 2 (January 2010), 37–71.
- [52] UHT, A. K., AND SINDAGI, V. Disjoint eager execution: An optimal form of speculative execution. In *Proceedings of the 28th International Symposium on Microarchitecture* (Ann Arbor, Michigan, United States, November 1995), pp. 313–325.
- [53] VATTIKONDA, B. C., DAS, S., AND SHACHAM, H. Eliminating fine grained timers in Xen. In *Proceedings of the ACM Workshop on Cloud Computing Security* (Chicago, Illinois, United States, October 2011), C. Cachin and T. Ristenpart, Eds., pp. 41–46.
- [54] VMWARE INC. *Understanding Memory Resource Management in VMware ESX Server*. Palo Alto, California, United States, 2009.
- [55] WALDSPURGER, C. A. Memory resource management in VMware ESX Server. In *Proceedings of the Fifth Symposium on Operating Systems Design and Implementation* (Boston, Massachusetts, United States, December 2002), D. E. Culler and P. Druschel, Eds., pp. 181–194.
- [56] WALTER, C. D. Longer keys may facilitate side channel attacks. In *Selected Areas in Cryptography* (2004), M. Matsui and R. J. Zuccherato, Eds., vol. 3006 of *Lecture Notes in Computer Science*, Springer-Verlag, pp. 42–57.
- [57] WEISS, M., HEINZ, B., AND STUMPF, F. A cache timing attack on AES in virtualization environments. In *Proceedings of the 16th International Conference on Financial Cryptography and Data Security* (Bonaire, February 2012), A. D. Keromytis, Ed.
- [58] XIAO, J., XU, Z., HUANG, H., AND WANG, H. A covert channel construction in virtualized environments. In *Proceedings of the 19th ACM Conference on Computer and Communication Security* (Raleigh, North Carolina, United States, October 2012), T. Yu, G. Danezis, and V. D. Gligor, Eds., pp. 1040–1042.
- [59] XU, J., KALBARCZYK, Z., AND IYER, R. K. Transparent run-time randomization for security. In *Proceedings of the 22nd International Symposium on Reliable Distributed Systems* (Florence, Italy, October 2003), pp. 260–269.
- [60] XU, Y., BAILEY, M., JAHANIAN, F., JOSHI, K., HILTUNEN, M., AND SCHLICHTING, R. An exploration of L2 cache covert channels in virtualized environments. In *Proceedings of the ACM Workshop on Cloud Computing Security* (Chicago, Illinois, United States, October 2011), C. Cachin and T. Ristenpart, Eds., pp. 29–40.

- [27] GNU隐私保护。http://www.gnupg.org, 2013年。
- [28] G ORDON, D. M. 快速指数运算方法综述。《算法杂志》27, 1 (1998年4月), 129-146。
- [29] G ullaach, D., B angerter, E., 与 K RENN, S. 缓存游戏——将基于访问的AES缓存攻击付诸实践。收录于《IEEE 安全与隐私研讨会论文集》(美国加利福尼亚州奥克兰, 2011年5月), 第490-595页。
- [30] H 宁格, N., 与 S HACHAM, H. 从随机密钥位重构RSA私钥。收录于《第29届国际密码学年会论文集 (crypto 2009)》(美国加利福尼亚州圣巴巴拉, 2009年8月), S. Halevi编, 第1-17页。
- [31] H ERMANN, M., 与 M AY, A. 模除除数求解线性方程: 关于任意比特的因式分解。发表于《密码学进展-亚洲密码2008》(澳大利亚墨尔本, 2008年12月), 《计算机科学讲义》第5350卷, 第406-424页。
- [32] H U, W.-M. 用模糊时间减少定时通道。收录于《IEEE 安全与隐私研讨会论文集》(美国加利福尼亚州奥克兰, 1991年5月), 第8-20页。
- [33] H UFFMAN, C. Windows 8与Windows Server 2012中的内存合并机制。http://blogs.technet.com/b/clinth/archive/2012/11/29/memory-combining-in-windows-8-and-windows-server-2012.aspx, 2012年11月。
- [34] 英特尔公司。《Intel 64和IA-32 Architecture Optimization Reference Manual》, 2012年4月。
- [35] 英特尔公司。《Intel 64和IA-32 体系结构软件开发人员手册第3A卷: 系统编程指南, 第1部分》, 2013年3月。
- [36] K IL, C., J UN, J., B ookholt, C., X U, J., AND N ING, P. 地址空间布局置换 (aslp): 商品软件的细粒度随机化研究。见《年度计算机安全应用会议论文集》(美国佛罗里达州迈阿密海滩, 2006年12月), 第339-348页。
- [37] K IVITY, A., K AMAY, Y., L AOR, D., L UBLIN, U., AND L IGUORI, A. kvm: Linux虚拟机监视器。收录于《Linux研讨会论文集》(加拿大安大略省渥太华, 2007年6月), 第一卷, 第225-230页。
- [38] K OCH, W. GnuPG 1.4.14 发布。http://lists.gnupg.org/pipermail/gnupg-announce/2013q3/000330.html, 2013年7月。
- [39] MIŁOŚ, G., murray, D. G., H 和, S., 以及 F 费特曼, M. A. 萨托里: 启发式页面共享。见《2009年USENIX年度技术会议论文集》(美国加利福尼亚州圣地亚哥, 2009年6月)。
- [40] M URPHY, D. L. 《TENEX 中的存储组织与管理》。载于《秋季联合计算机会议论文集, afips '72, 第一部分》(美国加利福尼亚州阿纳海姆, 1972年12月), 第23-32页。
- [41] O PEN SSL. http://www.openssl.org。
- [42] 有机化学家 E. I. 《Multics 系统: 结构分析》。麻省理工学院出版社, 1972年。
- [43] O SVIK, D. A., S HAMIR, A. 与 T ROMER, E. 缓存攻击与对策: 以AES为例。http://www.cs.tau.ac.il/~tromer/papers/cache.pdf, 2005年11月。
- [44] O WENS, R., 与 W ANG, W. 虚拟机中通过内存去重技术实现的非交互式操作系统指纹识别。收录于《IEEE 第30届国际高性能计算与通信会议论文集》(美国佛罗里达州奥兰多, 2011年11月), S. Zhong, D. Dou和Y. Wang编, IEEE出版社, 第1-8页。
- [45] 保罗尼, G. 如何在Intel IA-32和IA-64指令集架构上对代码执行时间进行基准测试。英特尔公司, 2010年9月。
- [46] P ercival, C. 缓存既无乐趣也无益处。http://www.daemonology.net/papers/htt.pdf, 2005。
- [47] R 是滕帕特, T., T ROMER, E., S HACHAM, H., 与 S AVAGE, S. Hey, 你, 别再烦我了: 探索第三方计算云中的信息泄露。收录于《第16届ACM计算机与通信安全会议论文集》(美国伊利诺伊州芝加哥, 2009年11月), E. Al-Shaer, S. Jha和A. D. Keromytis编, 第199-212页。
- [48] R IVEST, R. L., S HAMIR, A., 与 A DLEMAN, 一种获取数字签名和公钥密码系统的方法。《ACM通讯》21, 2 (1978年2月), 120-126。
- [49] S UZAKI, K., I IJIMA, K., Y AGI, T., 与 A RTHO, C. 内存去重对客户机的威胁。收录于《2011年欧洲系统安全研讨会论文集》(奥地利萨尔茨堡, 2011)。
- [50] PaX项目。http://pax.grsecurity.net/。
- [51] T ROMER, E. O SVIK, D. A. 与 S HAMIR, 《AES中的高效缓存攻击及对策》, 《密码学杂志》23, 2 (2010年1月), 37-71。
- [52] U HT, A. K. 与 S INDAGI, V. 不交并急切执行: 一种投机执行的最优形式。见《第28届国际微架构研讨会论文集》(美国密歇根州安娜堡, 1995年11月), 第313-325页。
- [53] V attikonda, B. C., D AS, S., AND S HACHAM, H. 在Xen中消除细粒度计时器。收录于《ACM云计算安全研讨会论文集》(美国伊利诺伊州芝加哥, 2011年10月), C. Cachin与T. Ristenpart编, 第41-46页。
- [54] VMWARE 公司 《VMware ESX Server内存资源管理指南》, 美国加州帕洛阿尔托, 2009年。
- [55] aldsurger, C. A. VMware ESX Server中的内存资源管理。收录于《第五届操作系统设计与实现研讨会论文集》(美国马萨诸塞州波士顿, 2002年12月), D. E. Culler与P. Druschel编, 第181-194页。
- [56] alter, C. D. 较长的密钥可能促进侧信道攻击。见《密码学精选领域》(2004年), M. Matsui和R. J. Zuccherato编, 《计算机科学讲义》第3006卷, Springer-Verlag出版社, 第42-57页。
- [57] W EISS, M., H EINZ, B., 与 S TUMPF, F. 虚拟化环境中AES的缓存定时攻击。收录于《第16届国际金融密码学与数据安全会议论文集》(博奈尔, 2012年2月), A. D. Keromytis主编。
- [58] X IAO, J., X U, Z., H UANG, H., 与 W ANG, H. 虚拟化环境中的隐蔽信道构建。收录于《第19届ACM计算机与通信安全会议论文集》(美国北卡罗来纳州罗利, 2012年10月), T. Yu, G. Danezis, V. D. Gligor主编, 第1040-1042页。
- [59] X U, J., K albarczyk, Z., 与 I YER, R. K. 透明运行时随机化技术在安全中的应用。收录于《第22届国际可靠分布式系统研讨会论文集》(意大利佛罗伦萨, 2003年10月), 第260-269页。
- [60] X U, Y., B AILEY, M., J ahanian, F., J OSHI, K., H iltonen, M., 与 S chlichting, R. 虚拟化环境中L2缓存隐蔽通道的探索。收录于《ACM云计算安全研讨会论文集》(美国伊利诺伊州芝加哥, 2011年10月), C. Cachin与T. Ristenpart编, 第29-40页。

- [61] ZHANG, Y., JULES, A., REITER, M. K., AND RISTENPART, T. Cross-VM side channels and their use to extract private keys. In *Proceedings of the 19th ACM Conference on Computer and Communication Security* (Raleigh, North Carolina, United States, October 2012), T. Yu, G. Danezis, and V. D. Gligor, Eds., pp. 305–316.

- [61] Z HANG, Y., J ULES, A., R EITER, M. K., 与 R istenpart, T. 交叉虚拟机侧信道及其用于提取私钥的应用。收录于 第19届ACM计算机与通信安全会议论文集（美国北卡罗来纳州罗利，2012年10月），由T. Yu、G. Danezis和V. D. Gligor主编，第305–316页。